

Table of Contents

Reference Vanilla Architecture for the Scalar Potential Language Model.....	2
Notation and Introductory Notes	2
Architecture Hyperparameters	2
State Variables.....	3
Learnable Parameters	4
Derived / Intermediate Quantities	4
Loss and Training Quantities	5
Oracle-Fit Quantities.....	5
Architectural Principles	6
A Note on the oracle fit.....	8
Training the SPLM : Learning the Reference Scalar Potential	9
One Scalar Potential for Every Layer (shared_potential_fit.py)	10
Symmetry as a Fingerprint of Conservative Flow (jacobian_symmetry.py).....	11
A Note on the Causal Probe (causal_probe.py).....	12
References	12
Appendix	13
Note on multi-ξ context channels and K-EMA.....	13
Role of ξ in the Euler-Lagrange step	13
Single channel : rank-1 causal mean	14
K-EMA : K causal weighted averages at several horizons	14
Note on the hidden width of the MLP $V\theta$	15
How MLP $V\theta$ consumes the bank of context channels ξtk	16
Structured and Gaussian $V\theta$: flatten, then project	16
Note on structured $V\theta$: Well Banks, Well Centres, Well Precisions, Well Mixture weights and their projections	17
Does the anisotropic Gaussian benefit beyond 4 Channels?	20
Can we do something better than flattening?	20
Depth conditioning and context banks in structured $V\theta$	21
Context banks	21
Depth codes as a per-layer shift.....	21
Where the forty wells sit before training.....	22
Module <code>splm_oracle_fit.py</code>	22
Function <code>fit_scalars</code>	22
Function <code>extract_oracle_tuples</code>	23
Module <code>model.py</code>	25
The SPLM Config : class <code>SPLMConfig</code>	25
Class <code>ScalarPotential</code>	25
Class <code>ScalarPotentialLM</code>	26
Putting it together: the reference forward pass.....	28
Module <code>model_sarf_mass.py</code>	29
The SARF Config - class <code>SPLMSARFMassConfig</code>	29
Function <code>causal_cumulative_mean</code>	29
Class <code>ScalarPotentialLMSARFMass</code>	29
Module <code>model_ln.py</code>	34
The Config Class: <code>SPLMSARFMassLNConfig</code>	34

Class ScalarPotentialLMSARFMassLN	35
Module train_splm.py	36
Per-mode configuration: build_config	36
Learning-rate schedule: lr_schedule	37
The training step	37
Module trajectory_extraction.py	38
Auto-detecting the checkpoint variant: _build_model_from_checkpoint	38
Extracting one trajectory: extract_one	39
Module shared_potential_fit.py	39
The shared scalar network: SharedPotential	40
Pooling samples across layers: build_samples	40
The joint fit: fit_shared_V	40
Module jacobian_symmetry.py	41
Unconstrained vs symmetric fit: fit_linear_and_symmetric	41
Why velocity matters: fit_second_order	42
Module token_direction_fit.py	42
Building token triples: build_token_triples	42
Module causal_probe.py	43
The probe: causal_violation_probe	43
Regression discipline: class_smoke	44
Probing natural transformers	44

Reference Vanilla Architecture for the Scalar Potential Language Model

Notation and Introductory Notes

Architecture Hyperparameters

The architecture hyperparameters are set at constructions and not learned. Here is a complete list of these hyperparameters

V – vocabulary size ; in code: `cfg.vocab_size`. vocabulary produced for example by the GPT-2 BPE Tokenizer).

d – embedding/hidden state dimension ; in code – `cfg.d`. Every tensor in SPLM lives in $\mathbb{R}^d$ or products of it thereof.

T – maximum sequence length (context window); in code `cfg.max_len`, example values: 128, 256, 512. Positional embeddings¹ are indexed for the positions $0, \dots, T - 1$.

L – number of integration steps (shared-weight depth). Each step applies the same V_θ, m, γ ; in code `cfg.L`.

Δt – integration time step for the damped Velocity-Verlet integrator; in code `cfg.dt`; Δt is fixed to 1.0.

d_V – hidden width of the V_θ MLP; in code `cfg.v_hidden`; $d_V = 128$ in smoke test code, 512 with TinyShakespeare dataset, 768 with TinyStories dataset.²

¹ In the classic (GPT-2-style) transformer architectures we have two separate embedding/encoding tables – **a)** *token embeddings* (TE) with shape (V, d) , indexed by token ID (tok_i) which encode *what* the symbol is, **b)** *positional encodings* (PE) with shape (T, d) , indexed by position $0 \dots T - 1$ which encode *where* in the sequence a slot sits. At the input layer **a)** and **b)** are combined additively: $x_i = TE[\text{tok}_i] + PE[i], i = 0, \dots, n - 1, n \leq T$

² See the Note on the MLP V_θ hidden width in the Appendix

n_v – number of hidden layers in the V_θ MLP (before the final scalar head); in code `cfg.v_dept`. $n_v = 2$ in the smoke test code and is 3 otherwise.

K_ξ – number of multi- ξ context channels.³ When the context permits, we will use K instead of K_ξ to denote number of context channels.

K_w, K_{mix} – number of energy wells per bank; K_{mix} represents the older notation adopted when the scalar potential was constructed using mixture models (e.g. SQ3)

n_{ctx} – number well banks; usually $n_{\text{ctx}} = K_\xi$

State Variables

State variables are computed during the forward pass

x – input token-id tensor; in code – **x**. Shape: (B, T) ; $x_{b,t} \in 0, \dots, V - 1$

$h_t^{(0)}$ – decontextualized initial embedding; we have $h_t^{(0)} = TE(x_t) + PE(t)$ ⁴; in code – **emb**. Shape: (B, T, d)

$h_t^{(\ell)}$ – hidden state at integration step $\ell \in 0, \dots, L$. Sometimes we drop the subscript t – h_ℓ . Each h_ℓ is a position in semantic space ($\mathbb{R}^d$). In code – **h**. Shape: (B, T, d)

v_ℓ – velocity at integration step $\ell \in 0, \dots, L$. $v_\ell := h_\ell - h_{\ell-1}$; in code – **v**. Shape: (B, T, d)

Disambiguation on the term *Hidden State*

In SPLM context and in the context the term *hidden state* denotes the d -dimensional residual stream vector $h_t^{(\ell)} \in \mathbb{R}^d$ associated with token position t at layer depth ℓ - i.e. row t of the per-layer activation tensor $H^{(\ell)} \in \mathbb{R}^{T \times d}$ that each sub-layer additively updates via $H^{(\ell)} = H^{(\ell-1)} + \text{SubLayer}(H^{(\ell-1)})$ in the residual-stream sense of Elhage et al 2021 ([8]). Because both the present SPLM framework and the Semantic Tube Prediction (STP) objective of Huang, LeCun & Balestrierio ([7]) speak of a “hidden-state trajectory”, it is essential to fix which axis of the tensor H the trajectory traverses. STP operates along the token axis: its hidden states are the final-layer representations $h_t^{(L)} = f(x_{\leq t})$, and the object it regularizes is the sequence-indexed path $\{h_t^{(L)}\}_{t=1}^T$, which the Geodesic Hypothesis posits to be locally linear and which the STP loss confines to a tubular neighborhood of a geodesic on the semantic manifold. The SPLM design, by contrast, operates along the depth axis: it studies the layer-indexed path $\{h_t^{(\ell)}\}_{\ell=1}^L$ of a *fixed* token position, treating depth ℓ as the time-like evolution parameter of a dynamical system whose equation of motion is the discretized Euler-Lagrange equation with a restoring force $-\nabla_h V_\theta$ from a learned scalar potential and layer-space dissipation $-\gamma v$. The two notions are therefore orthogonal projections of the same residual-stream tensor – STP constrains the inter-token geometry at the boundary layer $\ell = L$, whereas SPLM specifies the intra-token, depth-resolved dynamics for every ℓ - and the term “hidden state” should be read as referring to the common vector $h_t^{(\ell)}$, disambiguated only by the trajectory index (t for STP, ℓ for SPLM) under discussion.

ξ_t – per position causal context vector. In the vanilla SPLM: $\xi_t = \frac{1}{t} \sum_{s=1}^t h_s$ – causal cumulative mean, computed once at layer $\ell = 0$. In the SARF variant $\xi_t^{(\ell)} = \frac{1}{t} \sum_{s=1}^t h_s^{(\ell)}$ (re-pooled from **h.detach()** at each step ℓ). In code: **xi** / **xi_now**. Shape: (B, T, d)

f_ℓ – conservative force at integration step ℓ . $f_\ell = -\nabla_h V_\theta(\xi, h_\ell)$. In code: **f**. Shape: (B, T, d)

$\nabla_h V_\theta$ – gradient of the scalar potential w.r.t. the hidden state, computed via `torch.autograd.grad(V, h_in)`. The negation of this gradient is the conservative force **f**. In code: **grad_V**. Shape: (B, T, d)

³ See the Note on multi- ξ context channels in the Appendix

⁴ as with the previous footnote TE denotes token embedding and PE denotes positional encoding

Learnable Parameters

E – token embedding matrix; weights are tied: output logits are $h_L \cdot E^T$. In code: `self.E`. Initialized with $\mathcal{N}(0, 0.02^2)$. Shape: `nn.Embedding(V, d)`

$e_w = E_{w,:}$ - row of the embedding matrix for a row w (the embedding of a token w). Shape: `(d,)`

P – positional encoding/embedding matrix; Added to token embeddings as: $h_0 = E[x_t] + P_t$. In code: `self.P`. Initialized with $\mathcal{N}(0, 0.02^2)$. Shape: `nn.Parameter(T, d)`

θ – all weights and biases of the scalar-potential MLP. These parametrize the energy landscape $V_\theta(\xi, h)$. In code: `self.V_theta.net.parameters()`

$V_\theta(\xi, h)$ - the scalar potential (energy) function. An MLP: $\text{Linear}(2d \rightarrow d_V) \rightarrow \text{GELU} \rightarrow [\text{Linear}(d_V \rightarrow d_V) \rightarrow \text{GELU}] \times (n_V - 1) \rightarrow \text{Linear}(d_V \rightarrow 1)$; takes a concatenation $[\xi; h]$ and outputs a scalar energy. In code: `self.V_theta`. Shape: $\mathbb{R}^{2d} \rightarrow \mathbb{R}$

Note: the final layer of the V_θ MLP is initialized with very small weights ($\sigma = 0.002$) so the initial force is tiny, and the integrator starts stably.

m – semantic mass. Stored as a raw parameter; the physical mass is $m = \text{softplus}(m_{\text{raw}}) + \varepsilon$ to ensure positivity. Three modes in the SARF-mass variant: **(a)** *global* – single scalar shared by all tokens **(b)** *embed_head* - $m_t = \text{softplus}(\langle w_m, E[x_t] \rangle + b_m) + \varepsilon$ **(c)** *logfreq* - $m_t = \text{softplus}(m_{\text{raw}} + s_t) + \varepsilon$ where $s_t = -\log \hat{p}(x_t)$ is unigram surprisal. In code: `self.m / self.raw_m`. Shape: scalar or `(B, T, 1)`

γ – damping coefficient, controls the rate at which velocity decays. Given with $\gamma = \text{softplus}(\gamma_{\text{raw}})$. Can be learned or fixed. In code: `self.gamma / self.raw_gamma / cfg.fixed_gamma`.

α – logfreq scale factor for the surprisal-based mass mode, only present when `mass_mode = "logfreq"`. $\alpha = \text{softplus}(\alpha_{\text{raw}}^{\text{logfreq}})$. In code: `self.raw_logfreq_alpha`. Shape: scalar

Derived / Intermediate Quantities

$V_\theta(\xi_t, h_t^{(\ell)})$ - scalar potential energy evaluated at position $h_t^{(\ell)}$ with context ξ_t . In code: `self.V_theta(xi_now, h_in).sum()`. The `.sum()` aggregates over batch and position for a single backward pass; per-token gradients emerge because each h_t contributes independently to its own V_θ slice.

$v_{\ell+1} = \frac{v_\ell + \Delta t \cdot f_\ell / m}{1 + \Delta t \cdot \gamma}$ - velocity update via damped semi-implicit Euler (one half of the Velocity-Verlet scheme). In code: `v = (v + dt * f / m) / (1 + dt * gamma)`.

$h_{\ell+1} = h_\ell + \Delta t \cdot v_{\ell+1}$ - position update (second half of Verlet). In code: `h = h_in + dt * v`.

$\frac{1}{1 + \Delta t \cdot \gamma}$ - velocity retention – fraction of velocity surviving each step. At $\gamma = 1.0, \Delta t = 1.0$ the velocity retention is 0.5; at $\gamma = 2.4$ (Experiment A learned value) the velocity retention becomes 0.294.

LayerNorm projection - $h \leftarrow \frac{h - \mu}{\sigma + \varepsilon}$ after each step, projecting h to the unit-norm shell S^{d-1} (up to mean shift). Ensures compactness of the domain and a finite minimum of any continuous V_θ . In code: `self._project(h)`.

$\text{logits}_t = h_L^{(t)} \cdot E^T \in \mathbb{R}^V$ – Logits as defined in classical network architectures

Loss and Training Quantities

$\mathcal{L}_{\text{NTP}}$ – next token cross entropy loss. The only training objective in the vanilla SPLM – no auxiliary regularizers on V_θ , m , or γ . In code: `F.cross_entropy(logits, targets)`.

$\exp \mathcal{L}_{\text{NTP}}$ – The Perplexity (PPL) : it is our primary evaluation metric

Weight decay – AdamW parameter; it is L2 penalty on all parameters, In code: `weight_decay`. Typical values: 0.01 for Shakespeare, 0.0 for smoke.

Gradient clip – maximum gradient norm, in code: `grad_clip, nn.utils.clip_grad_norm_`, prevents integrator instability during early training, typical value: 1.0

LR Schedule – cosine annealing with linear warmup. In code: `lr_schedule(step, ...)`. Warmup: 20 (smoke) / 200 (Shakespeare) steps

Oracle-Fit Quantities

α_ℓ – per-layer scalar coefficient on V_θ in the oracle regression: $\Delta h_\ell \sim \alpha_\ell v_\ell + \beta_\ell (-\nabla_h V_\theta(\xi, h_\ell))$; $v_\ell := h_\ell - h_{\ell-1}$. Fitted by closed form least squares. In code: `alpha[ell]`.

β_ℓ – per-layer scalar coefficient on the conservative force $-\nabla_h V_\theta(\xi, h_\ell)$ in the oracle regression. Together $(\alpha_\ell, \beta_\ell)$ measure how much of the per-layer state change Δh_ℓ is explained by the model’s own learned potential. In code: `beta[ell]`.

R^2 – Shared-potential R^2 separator: fraction of variance in Δh_ℓ explained by the two-scalar model. Values near 1.0 indicate the dynamics is well-described by V_θ ; values near 0 indicate the model’s potential does not explain the layer-to-layer transitions.

H – raw hidden-state trajectory extracted from a single sentence, tensor with shape $(L + 1, T, d)$. In code: `H`.

GV – gradient of V_θ at each integration step i.e. $\nabla_h V_\theta(\xi_\ell, h_\ell)$. tensor with shape (L, T, d) . In code: `GV`.

$\Xi = [\xi_1, \dots, \xi_T]^T \in \mathbb{R}^{T \times d}$ – context vectors used during extraction. Tensor with shape (T, d) . In. code: `XI`.

Disambiguation of the context-vector subscript

Throughout the paper ξ carries a subscript whose meaning depends on which axis is being iterated:

- ξ_t – context indexed by *token position* $t \in \{1, \dots, T\}$. This is the natural index when the model processes a fixed layer step by step through the sequence; $\xi_t \in \mathbb{R}^{K \times d}$ is the K -channel context summary accumulated up to position t .
- ξ_ℓ – context indexed by *layer depth* $\ell \in \{1, \dots, L\}$. This is the natural index in the non-autonomous decomposition, where each layer has its own parameter block θ_ℓ and its own context slice ξ_ℓ drawn from the prefix; together they generate depth-indexed and context-indexed non-autonomy in attention-based transformers.

In both cases ξ denotes the same conceptual object – “whatever additional information V_θ receives beyond the hidden state h ” – and Ξ denotes the space from which it is drawn. The subscript signals only which axis is being varied; no new symbol is introduced.

Architectural Principles

Weight sharing across steps. V_θ, m, γ are the same at every integration step $\ell \in \{1, \dots, L\}$. This is what makes the architecture “conservative by construction” – the force law is a single shared scalar potential, not L different networks.

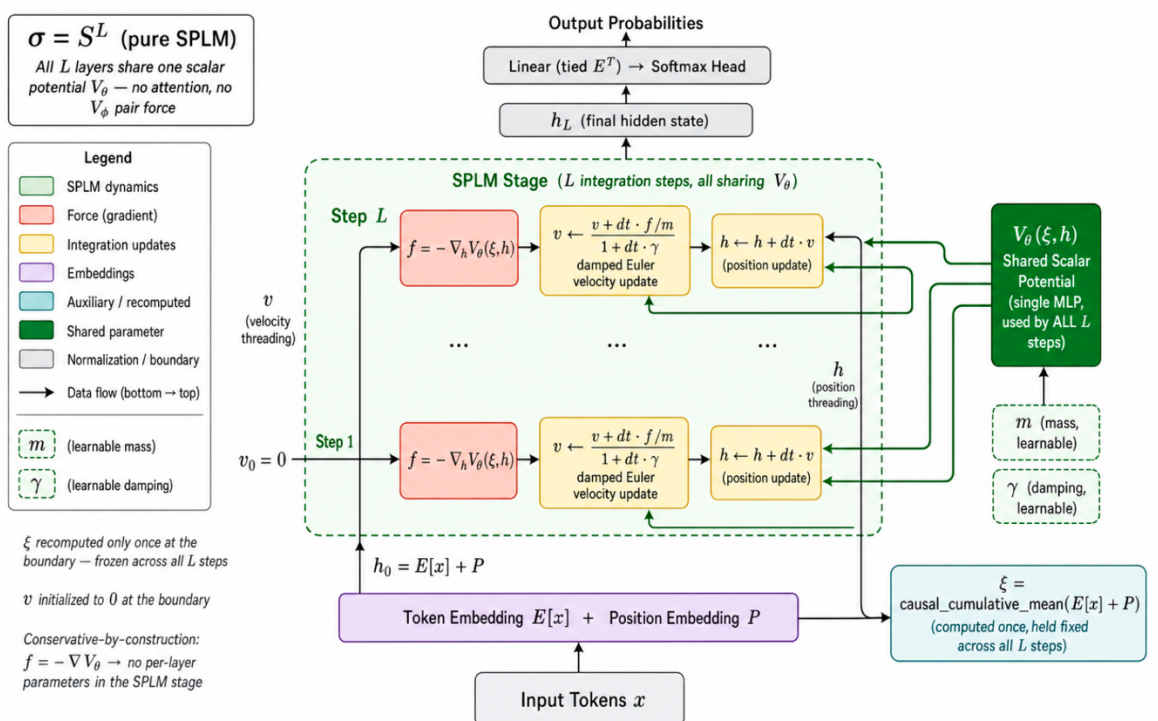
Asymmetric role of ξ vs h . In $V_\theta(\xi, h)$, ξ is a frozen conditional variable (detached from the graph in the SARF variant) while h is the differentiation variable. The force $f = -\nabla_h V_\theta$ is taken only w.r.t. h . This makes each $V_\theta(\xi_t, \cdot)$ a genuine scalar potential in h -space for fixed ξ_t .

Causal force fix (`causal_force = True`). In the SARF variant, ξ is computed from `h.detach()` to sever the autograd path from ξ back to h , eliminating an anti-causal gradient leak where $\frac{\partial V(t')}{\partial h(t)} \neq 0$ for $t' > t$.

ξ as a frozen conditioning variable. The conservative force $f_t = -\frac{\partial V_\theta(\xi_t, h_t)}{\partial h_t}$ is the partial derivative with respect to h at fixed ξ . This is what makes the force curl-free: $V_\theta(\xi, \cdot)$ is a genuine scalar potential landscape in h -space for each fixed ξ – different tokens see different slices $V_\theta(\xi_t, \cdot)$. In the base model, this is automatic: ξ is computed once from the initial embedding before the integration loop starts. In the SARF variant, ξ is re-pooled at every step from `h.detach()`, enforcing the same invariant dynamically. The `causal_force=True` config flag is the implementation switch that guarantees this.

Tied embeddings. The same matrix E is used for both input embedding and the output projection $h_L \cdot E^T$, reducing parameter count and coupling the input/output representations.

The reference (vanilla) architecture uses fixed- ξ context, a single scalar mass, no LayerNorm.



`model.py` – the base `ScalarPotentialLM` class

`model.py` – the `ScalarPotential` class

The class `ScalarPotential` obviously models $V_\theta(\xi, h)$. Recall, the conservative (curl-free) force is given with $f = -\nabla_h V_\theta(\xi, h)$.

$\text{Linear}(2d \rightarrow d_v) \rightarrow \text{GELU} \rightarrow [\text{Linear}(d_v \rightarrow d_v) \rightarrow \text{GELU}] \times (n_v - 1) \rightarrow \text{Linear}(d_v \rightarrow 1)$

So in the Shakespeare config, $2 \cdot 128 \rightarrow 512 \rightarrow 512 \rightarrow 512 \rightarrow 1$ with GELU nonlinearities, ending in a scalar head. The final 1-unit output makes V_θ a scalar field $\mathbb{R}^{2d} \rightarrow \mathbb{R}$, so that $-\nabla_h V_\theta(\xi, h)$ is a conservative (curl-free) force rather than an arbitrary vector field. Here the parameter θ refers to the MLP weights.

Thus, the signature of the scalar potential V is : **parameters** (the MLP weights) θ , **inputs** (ξ, h) .

ξ plays very specific asymmetric role among the two inputs which is central to the question why the architecture is conservative.

The force is given with $f = -\nabla_h V_\theta(\xi, h)$ and the gradient is taken only with respect to h , never ξ .

In the code: `torch.autograd.grad(V, h_in, ...)` differentiates w.r.t. `h_in` alone, and ξ is fed in detached (`h.detach()` before `causal_cumulative_mean`). So during the integration step, ξ is frozen – a *conditioning variable*, held constant while the differentiation on h is performed.

Thus, for a fixed ξ , the map $h \mapsto V_\theta(\xi, h)$ is an “honest” scalar potential and the step is pointwise conservative. ξ acts as a context-dependent *reshaping* of which scalar potential the token currently feels – different tokens (and, in

the re-pooled variants, different layers) see different slices $V_\theta(\xi, \cdot)$ of the same underlying two-argument function, but each slice is still a scalar field in h .

The scalar potential V_θ is an MLP whose learned weights are θ ; it takes two inputs (ξ, h) ; ξ is a frozen conditioning input that selects a potential slice while h is the differentiation variable whose negative gradient is the conservative force. $V_\psi(h)$ with no argument ξ cannot represent the ξ -conditioning. The missing conditioning is one of the two factors which are the source of the gap between the oracle's 0.931 and the perfect 1.0 on the LN variant.

Where is V_θ defined?

In [model_sarf_mass.py](#) (inherited unchanged by the LN model), `V_theta` is an attribute of the model, constructed once:

```
class ScalarPotentialLMSARFMass(nn.Module):
    def __init__(self, cfg):
        ...
        self.V_theta = ScalarPotential(cfg.d, cfg.v_hidden, cfg.v_depth)
```

and the ScalarPotential is the MLP

```
class ScalarPotential(nn.Module):
    def __init__(self, d, hidden, depth):
        layers = [nn.Linear(2 * d, hidden), nn.GELU()]
        for _ in range(depth - 1):
            layers += [nn.Linear(hidden, hidden), nn.GELU()]
        layers += [nn.Linear(hidden, 1)]
        self.net = nn.Sequential(*layers)
        ...
    def forward(self, xi, h):
        return self.net(torch.cat([xi, h], dim=-1))
```

The learnable weights here - `self.V_theta.net.parameters()` are θ .

A Note on the oracle fit

The oracle ([splm_oracle_fit.py](#)) is the ordinary shared-potential regression with one substitution – instead of training a fresh generic $V_\psi(h)$, it plugs in the model's own learned $V_\theta(\xi, h)$ and fits only the two per-layer scalars by closed-form least squares (function `fit_scalars` – see Appendix)

$$\Delta h_\ell \approx \alpha_\ell v_\ell - \beta_\ell \nabla_\ell V_\theta(\xi_\ell, h_\ell), \quad v_\ell := h_\ell - h_{\ell-1}$$

Two observations from `extract_oracle_tuples` (shown in the Appendix):

The gradient regressor G_ℓ (`GV[1]` in the code) is not a freshly trained generic potential: it is `grad_h V_theta(xi, h_1)`, the gradient of the model's own learned scalar potential, evaluated at the exact hidden states h_1 the model visits when it replays the sentence. The oracle does not re-learn an energy landscape — it reuses the one the network already learned and only asks how well a single per-layer rescaling of that landscape's force reproduces the observed motion.

The velocity regressor is reconstructed by finite differences. In `build_pools` the per-step velocity proxy is `v_1 = H[1] - H[1-1]` and the regression target is the next displacement `Delta_h_1 = H[1+1] - H[1]`. These are exactly the two ingredients of the damped semi-implicit Euler update, so the two-scalar ansatz `Delta_h_1 ~ alpha_1 *`

$v_l - \beta_l * \text{grad}_h V_{\theta}(x_i, h_l)$ is not an approximation of the dynamics — it is the dynamics, written with the integrator constants left free.

To see why, expand the update used during extraction. With $v_{l+1} = (v_l + dt * f_l / m) / (1 + dt * \gamma)$ and $f_l = -\text{grad}_h V_{\theta}$, the displacement is

$$\Delta h_l = dt * v_{l+1} = \left(\frac{1}{1 + dt * \gamma} \right) * v_l - \left(\frac{dt^2}{m * (1 + dt * \gamma)} \right) * \text{grad}_h V_{\theta},$$

where v_l proxy = $dt * v_l$. This is precisely $\alpha_l * v_l + \beta_l * (-\text{grad}_V)$ with the closed-form values $\alpha_l = 1 / (1 + dt * \gamma)$ and $\beta_l = dt^2 / (m * (1 + dt * \gamma))$. The least-squares step in `fit_scalars` simply recovers these two constants per layer.

Consequently the oracle is an upper bound, not a model. For the reference (vanilla) architecture, where x_i is held fixed and no projection is applied, the displacement is an exact linear combination of the two regressors and the fit is numerically perfect: `splm_oracle_fit.py` returns $R^2 = 1.0000$ at every layer. That figure is the right consistency check — it confirms that the extracted (H, GV, XI) tuples and the integrator algebra agree to machine precision. The LayerNorm-after-step variant (`model_ln.py`) lands lower, at $R^2 \sim 0.931$. The shortfall has a single structural cause: after every step the state is projected back onto the unit-LayerNorm shell, $h_{l+1} \leftarrow (h_{l+1} - \mu) / (\sigma + \epsilon)$, a non-affine map. The observed displacement is then $\Delta h_l = \text{project}(h_l + dt * v_{l+1}) - h_l$, which is no longer an exact linear combination of v_l and $-\text{grad}_V$; the per-layer two-scalar model captures the projected motion only up to the curvature of the shell, and that residual is what separates 0.931 from 1.0. The gap is the price the LN variant pays for a guaranteed finite energy minimum (see Module `model_ln.py`), not evidence that its force is any less conservative.

Read together, the two numbers bracket the question the shared-potential separator asks. The oracle (`true V_theta(x_i, h)`: $R^2 = 1.0000$ base / 0.931 LN) is the upper bound reachable when the regressor is the model's own context-conditioned potential. The step-2 fit of a generic, context-free $V_{\psi}(h)$ shared across layers sits below it: dropping the x_i argument removes the per-token slice selection that V_{θ} exploits, so a single $V_{\psi}(h)$ cannot represent the context-dependent reshaping of the landscape. The distance between the generic- V_{ψ} fit and the oracle measures exactly how much of SPLM's per-layer motion is context-driven rather than position-in-state-space-driven.

Training the SPLM : Learning the Reference Scalar Potential

The reference scalar potential is learned end-to-end with a single objective and no trajectory-level supervision. The base model `ScalarPotentialLM` is trained by `train_splm.py`; the SARF-mass and LayerNorm variants by `train_splm_em_ln.py` ([4]). In every case the only loss is next-token cross-entropy on the tied-embedding logits $\text{logits} = h_L @ E^T$. There is no auxiliary penalty on the velocity, the potential, the per-step displacement, or the trajectory shape: the energy landscape V_{θ} , the mass m , and the damping γ are shaped solely by the requirement that the depth- L damped flow carry the initial embedding h_0 to an endpoint h_L whose logits predict the next token.

What makes this non-trivial is that the loss gradient must pass through the force computation at every step. Inside `integrate`, the force is itself a gradient: `grad_V, = torch.autograd.grad(V, h_in, create_graph=self.training, retain_graph=True)`. The flag `create_graph=self.training` keeps that inner gradient inside the autograd graph during training, so backpropagating the cross-entropy loss differentiates through the gradient of V_{θ} — a second-order (double-backward) computation. The optimizer therefore receives $dL/d\theta$ terms that already account for how a change in the potential reshapes the force field at each of the L shared steps. This is the mechanism by which a single shared V_{θ} learns a genuine force law, rather than L independent layers learning arbitrary maps.

The optimization recipe is deliberately plain. The optimizer is AdamW (betas (0.9, 0.95) in the em_ln trainer) with weight decay 0.01 for the Shakespeare runs and 0.0 for smoke. The learning rate follows a cosine schedule with linear warmup (lr_schedule): 20 warmup steps (smoke) or 200 (Shakespeare), then cosine decay to zero. Gradients are clipped to a global norm of grad_clip = 1.0 (nn.utils.clip_grad_norm_) before every step — this clip matters early in training, when the integrator can briefly amplify gradients before v_theta has settled into a stable basin. Training runs for 300 steps (smoke) or 4000 steps (Shakespeare). The core loop is:

```
for step in range(train_cfg["steps"]):
    lr_now = lr_schedule(step, lr, warmup_steps, steps)
    for g in optim.param_groups:
        g["lr"] = lr_now
    x, y = get_batch(train_ids, batch_size, block_size, rng)
    _, loss = model(x, y) # forward = L damped EL steps + CE loss
    optim.zero_grad(set_to_none=True)
    loss.backward() # double-backward through the force
    grad_norm = nn.utils.clip_grad_norm_(model.parameters(), grad_clip)
    optim.step()
```

The mass and damping are optimized jointly with the potential, parametrized through softplus so they remain positive: $m = \text{softplus}(\text{raw_m}) + 1e-3$ and $\gamma = \text{softplus}(\text{raw_gamma})$. For the damping sweep the damping is instead pinned to a constant via `cfg.fixed_gamma`, in which case `raw_gamma` is frozen and the `gamma` property returns that fixed value. Experiment A's learned damping settles near $\gamma \sim 0.30$, giving a per-step velocity retention $1/(1+dt*\gamma) \sim 0.77$ at $dt = 1.0$ — the flow keeps most of its momentum between layers rather than collapsing immediately onto the nearest minimum.

At the end of training the checkpoint stores the full config (`model_cfg`), the state dict, the final validation loss / perplexity, the final gamma, and a variant tag ("sarf_mass_ln" for the LN model). The tag is what later lets the extractors and the oracle (`splm_oracle_fit.py`, `extract_oracle_tuples`) reconstruct the correct subclass and replay the exact dynamics that produced the trained potential.

One Scalar Potential for Every Layer (`shared_potential_fit.py`)

Everything up to this point has described how to *build* a conservative-by-construction language model and how to *train* it. The natural next question is diagnostic rather than constructive: given a trajectory of hidden states — from our model or from a black-box transformer — how do we *test* whether its dynamics is genuinely the gradient flow of a single energy? `shared_potential_fit.py` answers the strictest possible version of that question.

The test is deceptively simple to state. Pool, across every interior layer, every token and every sentence, the triple of centered position x_l , velocity proxy $v_l = x_l - x_{l-1}$ and one-step displacement $\Delta x_l = x_{l+1} - x_l$. Then fit a single neural scalar $V_{\text{psi}}(x)$, shared across all layers, together with per-layer scalars α_l , β_l , to the ansatz $\Delta x_l \approx \alpha_l v_l - \beta_l \text{grad } V_{\text{psi}}(x_l)$. The interesting word is *single*. A per-layer symmetric operator (the subject of the next section) is a weak constraint; demanding that one smooth scalar explain twelve layers of behaviour at once is a far stronger one, and it is exactly the property that distinguishes a true physical potential from a layer-wise curve-fit.

Two design choices make the result trustworthy rather than merely suggestive. First, the per-layer scalars are initialised to zero, so the fit *begins* as the static null $\Delta x = 0$ and has to earn every fraction of explained variance:

```
alpha = nn.Parameter(torch.zeros(L)) # start as the static null ...
beta  = nn.Parameter(torch.zeros(L)) # ... and earn every bit of variance
```

```
# ...
y_pred = alpha[lb].unsqueeze(-1) * vb - beta[lb].unsqueeze(-1) * gV
loss = F.mse_loss(y_pred, yb) + 1e-5 * torch.mean(gV ** 2)
```

Second, the script reports three *nested* baselines — static null, velocity-only $\alpha_1 v_1$, and the full velocity-plus-shared- v fit — each a strict subset of the next. Because they are nested, any gain the shared potential shows over the velocity-only baseline cannot be an artifact of momentum; it is attributable to the conservative force and nothing else. The small $1e-5 * \text{mean}(gV^2)$ penalty keeps the learned potential from inflating its own gradient to cheat the residual.

The deepest insight is in the asymmetry of what we expect. Our SPLM passes this test *by construction* — its dynamics literally is the gradient of one scalar v_{theta} — so a high held-out R^2 there is a positive control confirming the diagnostic itself is sound. The scientific payload is what happens to a *pretrained transformer*: does the composition of attention, MLP and LayerNorm across all layers admit any single smooth potential, or does it not? Reporting R^2 on held-out sentences, rather than the training pool, turns this from a curiosity into a falsifiable claim about whether attention dynamics is conservative at all.

Symmetry as a Fingerprint of Conservative Flow (`jacobian_symmetry.py`)

If the shared-potential fit is the strict global test, `jacobian_symmetry.py` is the sharp local one. It rests on a single fact from mechanics: the linearisation of a gradient flow is the Hessian of a scalar, and Hessians are symmetric. So if we fit a per-step linear operator M_1 to the transition $x_{\{1+1\}} - x_1 \approx M_1 x_1$ on a principal-component subspace, then constraining $M_1 = M_1^T$ should cost essentially nothing whenever the underlying flow is conservative. Symmetry is a *fingerprint*: a necessary signature that a hidden scalar energy is in charge.

The most interesting idea in the module is a methodological warning about how this test can be done *wrong*. A naive position-only fit $\Delta x \approx M x$ is confounded for second-order damped dynamics, because a single step mixes the position x_1 with the hidden velocity $v_1 \approx x_1 - x_{\{1-1\}}$. That mixing can manufacture apparent asymmetry even in a flow that is genuinely conservative — meaning a careless 'GPT-2 looks asymmetric' conclusion drawn from a position-only fit could be an artifact of the diagnostic, not a property of the model. The fix is to make the fit velocity-aware, regressing on the concatenated design $[x \mid v]$ so that an unconstrained A absorbs the velocity term while M — now the clean per-step spring matrix — is the only operator whose symmetry is judged:

```
# position-only (confounded for 2nd-order dynamics): y = M x
# velocity-aware (the clean test):
Phi = np.concatenate([Xc, Vc], axis=1) # [X | V]
B = np.linalg.solve(Phi.T @ Phi + reg, Phi.T @ Yc).T
M_full, A_full = B[:, :k], B[:, k:] # M is the spring matrix; A absorbs v
```

There is also a small piece of implementation elegance worth pointing out. Rather than running a constrained optimiser to enforce $M = M^T$, the module parameterises M by the free entries of its upper triangle and builds a design matrix in which each off-diagonal parameter (i, j) feeds *both* output channel i (through predictor j) and channel j (through predictor i). The symmetry constraint then falls out of an ordinary least-squares solve — no projection, no iteration.

What makes the diagnostic decisive is the size of the contrast it produces. For the attention transformers in the companion failure study, the unconstrained fit reaches R^2 in $[0.5, 0.8]$ while the symmetric fit goes negative — worse than predicting the mean displacement. That is not a model that is slightly asymmetric; it is a flow dominated by a non-Hessian, rotational component that no scalar potential can ever reproduce. Our SPLM, run

through the identical code, is expected to show the symmetric fit tracking the unconstrained one layer by layer — the fingerprint of the conservativity it was built to have.

A Note on the Causal Probe (`causal_probe.py`)

The final diagnostic guards against the subtlest failure mode of the entire construction, and its story is the most cautionary. Recall that the conservative force is the gradient of a sum of potentials over positions, $f(t) = -d(\sum_{t'} V[t']) / dh[t]$. The moment the context vector x_i is built as a causal running average that itself includes $h[t]$, differentiating that summed energy picks up off-diagonal terms $dV[t'] / dh[t]$ for $t' > t$. In other words, the very elegance that makes the architecture physical — force as the gradient of a global energy — quietly opens an *anti-causal channel*, and a trained v_{θ} will happily learn to route information backwards through it, leaking future tokens into past predictions and producing a 'too good to be true' validation loss that is, in fact, cheating.

The insight worth dwelling on is that this bug is not a coding slip; it is a structural hazard specific to energy-based, differentiate-the-objective architectures. A standard decoder never differentiates a summed loss to produce its forward activations, so it cannot leak this way. The fix is a single token — compute x_i from $h.detach()$, severing the autograd path from the context back to the state — and it is exposed everywhere as a `causal_force: bool = True` config field:

```
# leak:  xi = causal_mean(h)          -> d(sum_t' V[t'])/dh[t] picks up t' > t
# fix:   xi = causal_mean(h.detach())  # sever the anti-causal autograd path
```

`causal_probe.py` converts that fix into a falsifiable, exact test. It perturbs a single token at position `t_pert`, runs two forward passes, and measures the largest change in any logit strictly before the perturbation. For a correct autoregressive model this number must be zero — not small, but zero to machine precision. Seeding the buggy and fixed models identically so they share weights isolates the entire difference to the detach:

```
pre_f, _, _ = causal_violation_probe(model_fixed, vocab_size=257)
ok_fixed = pre_f < 1e-6      # the fixed model MUST be causal to machine precision
```

Two things elevate this from a unit test to a trust anchor. First, the same probe is run unchanged on natural transformers — a local MatchedGPT checkpoint, or gpt2 and EleutherAI/pythia-160m from the Hub — where the causal-side delta is zero by construction of the attention mask. Holding our physics-style model to the identical standard as a vanilla decoder is what licenses comparing their perplexities at all. Second, with `--strict` the probe exits non-zero on any violation, so it can sit in a continuous-integration gate. The result is a clean separation between belief and proof: the headline efficiency and perplexity numbers elsewhere in this document are quotable precisely because this probe certifies that the conservative dynamics never once looked at the future.

References

- [1] Gueorguiev, D. (2026). Locally Conservative, Globally Not: Diagnosing the Lagrangian Structure of Pretrained Transformers. Preprint: Zenodo & SSRN. GitHub: https://github.com/dimitarpg13/paper_tmlr_1
- [2] Gueorguiev, D. (2026). Semantic Simulation: A Prescriptive Lagrangian Framework for Efficient Semantic Inference. Preprint: Zenodo & SSRN. GitHub: github.com/dimitarpg13/semsimula-paper
- [3] https://github.com/dimitarpg13/paper_tmlr_1/blob/main/notebooks/conservative_arch/model_sarf_mass.py

- [4] https://github.com/dimitarpg13/paper_tmlr_1/blob/main/notebooks/conservative_arch/ln_damping_sweep/train_splm_em_ln.py
- [5] https://github.com/dimitarpg13/paper_tmlr_1/blob/main/notebooks/conservative_arch/splm_oracle_fit.py
- [6] https://github.com/dimitarpg13/paper_tmlr_1/blob/main/notebooks/conservative_arch/train_splm.py
- [7] https://github.com/dimitarpg13/paper_tmlr_1/blob/main/notebooks/conservative_arch/trajectory_extraction.py
- [8] https://github.com/dimitarpg13/paper_tmlr_1/blob/main/notebooks/conservative_arch/shared_potential_fit.py
- [9] https://github.com/dimitarpg13/paper_tmlr_1/blob/main/notebooks/conservative_arch/jacobian_symmetry.py
- [10] https://github.com/dimitarpg13/paper_tmlr_1/blob/main/notebooks/conservative_arch/token_direction_fit.py
- [11] https://github.com/dimitarpg13/paper_tmlr_1/blob/main/notebooks/conservative_arch/causal_probe.py
- [12] [Byte-Pair Encoding Tokenization, LLM Course, Hugging Face](#)
- [13] [Semantic Tube Prediction : Beating LLM Efficiency with JEPA, Hai Huang, Yann LeCunn, Randall Balestriero, 2026](#)
- [14] [Mathematical Framework of Transformer Circuits, Nelson Elhage et al, Anthropic, 2021](#)
- [15] [Analytic Multi-Channel Integration in a Single Structured \$V_\theta\$ Well, companion note to \[2\], 2026](#)
- [16] [Gaussian Mixture Model Clearly Explained, R. Ravihara, Towards Data Science, 2023](#)
- [17] [The Elements of Statistical Learning, Section 8.5: *The EM Algorithm* and Section 12.7: *Mixture Discriminant Analysis*, T. Hastie, R. Tibshirani, J. Friedman, print 12, 2017](#)

Appendix

Note on multi- ξ context channels and K-EMA

Role of ξ in the Euler-Lagrange step

At each integration layer the hidden state $h_t \in \mathbb{R}^d$ of token t is driven by a conservative force from a scalar potential that is allowed to depend on a causal context summary ξ_t :

$$m_t \ddot{h}_t + \gamma \dot{h}_t = -\nabla_{h_t} V_\theta(\xi_t, h_t) \quad (\text{a1.ode})$$

(In the related models PARFLM, Fock-PARFLM there are additional pair and Fock terms which do not replace ξ .)

Without ξ , $V_\theta(\cdot, h_t)$ would be a *memoryless* well: token t would feel only its own current state. ξ is the mechanism by which *past hidden states* enter the single-particle potential while remaining a *fixed external field* for the h_t -gradient (so the force stays causal and conservative in h_t).

In the SARF-faithful stack, ξ is recomputed at every layer from the current h -stream, not frozen from layer-0 embeddings. Implementation: `causal_force=True` feeds `h.detach()` into the ξ block so autograd cannot route future V_θ values back into past tokens (that is the anti-causal leak that collapsed pre-fix multi- ξ PPL).

Single channel : rank-1 causal mean

The original SPLM / SARF construction uses one context vector, the causal cumulative mean

$$\xi_t = \frac{1}{t} \sum_{s \leq t} h_s \quad (\text{ccm.1})$$

(`causal_cumulative_mean` in `sarf_variant/model_sarf.py`)

Then,

$$V_\theta : \mathbb{R}^{2d} \rightarrow \mathbb{R}, \text{input} = \text{cat}(\xi_t, h_t)$$

This is a *rank-1 summary* of the prefix : any two prefixes with the same arithmetic mean of hidden states are indistinguishable to V_θ . Attention, by contrast, mixes the prefix in a content-conditioned way. The single- ξ bottleneck (demonstrated by experiment E11/R6 ladder available in the companion repo) is the structural reason the multi-channel extension exists.

K-EMA : K causal weighted averages at several horizons

Multi- ξ replaces the one mean with K parallel exponential kernels (`MultiChannelXi` in `multixi/model_multixi.py`). Channel k has a decay $\alpha_k \in (0,1)$:

$$W_k[t, s] = \frac{\alpha_k^{t-s}}{\sum_{r \leq t} \alpha_k^{t-r}} \quad (s \leq t) \quad , \quad W_k[t, s] = 0 \quad (s > t) \quad , \quad \xi_t^{(k)} = \sum_{s \leq t} W_k[t, s] h_s \quad (\text{chk.1})$$

Weights in (chk.1) are built with in log-space and are row-normalized with `logsumexp` , so each row sums to 1 even for α near 0 or 1.

Endpoint interpolation (this is the implemented kernel, not the classical IIR EMA):

- $\alpha_k \rightarrow 0$: $W_k = I$, so $\xi_t^{(k)} = h_t$ (no past);
- $\alpha_k \rightarrow 1$: W_k is the causal uniform mean , so $\xi_t^{(k)}$ recovers the single- ξ baseline;
- $\alpha_k \in (0 + \varepsilon, 1 - \varepsilon)$ where $\varepsilon > 0$ is small enough but fixed : a causal weighted mean with effective horizon $\tau_k \sim 1/(1 - \alpha_k)$.

The architecture therefore is strict superset : driving $\alpha_k \rightarrow 1$ recovers the old context; other channels can keep shorter or longer memory.

Note that this interpolation (chk.1) is not the same as the textbook recurrence

$$\xi_t = \alpha \xi_{t-1} + (1 - \alpha) h_t \quad (\text{iir.1})$$

That IIR form is unnormalized and its limit $\alpha \rightarrow 1$ limit freezes early tokens rather than recovering the cumulative mean. In [2], (iir.1) is denoted as R6.h.0. With (iir.1) when $\alpha_k \rightarrow 0$ $\xi_t^{(k)} = h_t$ like with (chk.1), however when $\alpha_k \rightarrow 1$ $\xi_t^{(k)} \approx \xi_{t-1}^{(k)}$ (memory freezes).

The code in the companion repo uses the row-normalized kernel (chk.1) above with $\alpha_k = \text{sigmoid}(\text{raw}_k)$ which guarantees $\alpha_k \in (0,1)$. The decays α_k can be frozen or learnable (xi_learnable). All current models in the SPLM family use learnable decays.

Default Configuration : $K = 4$ (config field xi_channels)

Init mode	Typical α	Horizons (tokens, $\tau \sim 1/(1 - \alpha)$)
Explicit (R6.h.0)	0, 0.5, 0.9, 0.99	$\approx 1, 2, 10, 100$
Log-spaced (R6.h.1))	$\alpha_k = 1 - \tau_{\max}^{-\frac{k}{(K-1)}}, \tau_{\max} = 100$	$\approx 1, 4.6, 21.5, 100 \Rightarrow \alpha \approx 0., 0.785, 0.954, 0.99$

Log-spacing is the prior that relevant linguistic timescales are scale-free up to a paragraph length: horizons sit on a geometric grid rather than on an ad-hoc list.

Forward shape: $h \in \mathbb{R}^{B \times T \times d} \mapsto \xi \in \mathbb{R}^{B \times T \times d \times k}$.

Note on the hidden width of the MLP V_θ

This note is on the difference between the hidden width of MLP V_θ denoted with d_v and the number of hidden dimensions of the embeddings (or state) d . d_v and d represent different widths for two different objects. The hidden state dimensions width d represents the fact that each semantic particle (vector embedding of a token) lives in $\mathbb{R}^d$ (the current TinyStories scale-up model code uses $d = 256$; some of the previous experiments used $d = 192$ and $d_v = 768$). d_v is the MLP width of the scalar potential $V_\theta(\xi, h)$. That network is not a residual stream and it maps a concatenated input to one energy:

```
layers: List[nn.Module] = [nn.Linear(2 * d, hidden), nn.GELU()]
...
layers += [nn.Linear(hidden, 1)]
def forward(self, xi: torch.Tensor, h: torch.Tensor) -> torch.Tensor:
    return self.net(torch.cat([xi, h], dim=-1))
```

So the first layer already sees the concatenated pair (ξ, h) of dimension $2d$ in the single-context `ScalarPotential`, and dimension $(K_\xi + 1)d$ once the K-EMA bank is concatenated with h in `ScalarPotentialMultiXi`. The quantity d_v (`v_hidden`) is the width of the GELU stack that follows that first map, not the embedding width d .

The first linear map of `ScalarPotential` therefore has shape $2d \rightarrow d_v$. In the TinyStories scale-up this is a map $512 \rightarrow 1024$ when $d = 256$ and $d_v = 1024$. That is only a factor of two above the concatenated input, which is an ordinary expansion for a feed-forward network, not an accidentally oversized hidden layer. The multi-channel module `ScalarPotentialMultiXi` widens only that first map, to $(K_\xi + 1)d \rightarrow d_v$, and leaves the rest of the GELU stack at width d_v . How the K-EMA bank is consumed is taken up in the next subsection; the point of the present note is why d_v itself is chosen larger than d .

The scalar potential is asked to define a complicated landscape on $\mathbb{R}^d$: many wells, saddles, and a context-dependent force $f = -\nabla_h V_\theta$. A network that mapped $2d$ (or $(K_\xi + 1)d$) down to a hidden width of order d and then

to a single energy would be a severe bottleneck. Universal-approximation heuristics for a single hidden layer say that the hidden width should be at least the input dimension if the target function is allowed to be even moderately oscillatory. Here the target is worse than a generic function of h : it must also change with ξ , and the integrator trains through the gradient $\nabla_h V_\theta$ rather than through the scalar value alone. If d_v is comparable to d , that gradient tends to be too smooth, in effect low-rank as a map of h , and the force cannot steer particles into distinct semantic basins.

The same expansion appears in a Transformer for a different reason. Self-attention works at width d , but the position-wise feed-forward sublayer typically expands to $4d$ before writing back. In the SPLM family V_θ is the analogue of that feed-forward: it is the learned nonlinearity that shapes the trajectory of h . Setting d_v to about $3d$ or $4d$ is therefore following an established capacity pattern, not matching the embedding width for a misplaced sense of fairness. Typical pairings in this repository are:

Setting	Embedding width d	MLP width d_v (v_hidden)	Ratio d_v/d
Smoke tests	64	128	2
Tiny Shakespeare	128	512	4
TinyStories (SARF-mass)	192	768	4
TinyStories scale-up (E9)	256	1024	4

So $d_v > d$ is the expected operating point. Setting $d_v = d$ is not a parameter-matched control of the landscape; it usually underfits the potential, and the force that the integrator sees is too impoverished to be compared with a $4d$ run on equal terms. The Helmholtz H1.5 cells that narrowed v_hidden did so as a FLOP experiment, not as a claim that the embedding width is the natural width of V_θ .

How MLP V_θ consumes the bank of context channels $\xi_t^{(k)}$

The original energy is defined as MLP (GELU stack) on the concatenated pair (ξ, h) . Multi- ξ only widens the first linear map:

$$V_\theta : \mathbb{R}^{(K+1)d} \rightarrow \mathbb{R}, \text{ input} = \text{cat}(\xi_t^{(1)}, \dots, \xi_t^{(K)},)$$

Thus, ScalarPotential is $2d \rightarrow v_{\text{hidden}} \rightarrow \dots \rightarrow 1$, while ScalarPotentialMultiXi is $(K+1)d \rightarrow v_{\text{hidden}} \rightarrow \dots \rightarrow 1$. v_{hidden} (in code v_hidden) typically $3d - 4d$ e.g. With TinyStories we set $d = 256$, $v_{\text{hidden}} = 1024$. The α_k 's are K scalars.

Because the MLP hidden layers mix all $(K+1)d$ coordinates, they can in principle use *cross-horizon correlations* (short-timescale ξ vs long-timescale ξ vs current h). That is why the MLP reference on TinyStories benefits from $K_\xi = 4$ relative to single- ξ whereas a diagonal-structured well bank often does not (in [2] : doubling K_ξ from 4 to 8 did not help SQ3; the MLP baseline at $K_\xi = 4$ reached lower PPL compared to its structured counterpart at $K_\xi = 8$).

Structured and Gaussian V_θ : flatten, then project

Structured potentials (quadratic (SQ3), isotropic and anisotropic Gaussians) were originally designed for single context vector of dimension ξ_d . The multi- ξ adapters do not use new geometry to incorporate the multiple channels in the well – they simply flatten the bank:

$$\xi_{\text{flat}} = \text{reshape}(\xi) \in \mathbb{R}^{K\xi^d}, V = V_{\text{inner}}(\xi_{\text{flat}}, h)$$

- `StructuredVThetaMultiXiAdapter` – quadratic / LSE mixtures ; inner `xi_d = K_xi*d` in code.
- `GaussianVThetaMultiXiAdapter` – same flatten into `mu_proj`, `a_proj`, and `w_proj`.

Well centers (`mu_proj`), precisions (`a_proj`), and mixture weights (`w_proj`) then become linear functions of the concatenated multi-resolution summary. A later variant, `MultiContextGaussianVTheta`, gives each $\xi^{(k)}$ its own well bank and sums $V = \sum_k V_k(\xi^{(k)}, h)$, so attractors are not forced to read a blurred $K_\xi d$ vector. That is an expressivity choice on top of K-EMA, not a change to the EMA itself. Specifically, the k -th well bank $V_k(\xi^{(k)}, h)$ in `MultiContextGaussianVTheta` is given as

$$V_k(\xi^{(k)}, h) = -\sum_{j=1}^{K_w} W_j^{(k)}(\xi^{(k)}, h) \quad (\text{mcgvt.1})$$

The j -th term $W_j^{(k)}(\xi^{(k)}, h)$ in the RHS of (mcgvt.1) represents a single Gaussian well:

$$W_j^{(k)}(\xi^{(k)}, h) = w_j^{(k)}(\xi^{(k)}) \exp\left(-\frac{1}{2}(h - \mu_j^{(k)})^T P_j^{(k)}(h - \mu_j^{(k)})\right) \quad (\text{mcgvt.2})$$

Finally, the total structured potential V_θ , abbreviated with V , is the sum of all banks:

$$V = \sum_{k=1}^{K_\xi} V_k(\xi^{(k)}, h) \quad (\text{mcgvt.3})$$

Note on structured V_θ : Well Banks, Well Centres, Well Precisions, Well Mixture weights and their projections

Definition Bank

Bank is one copy of structured well module: a mixture model (e.g. GMM⁵) with its own projections, producing one scalar potential from one context vector ξ and hidden state h .

The number of wells in the bank is denoted with K_w in general. In older companion notes and in [2] in the context of using mixture models to construct V_θ we use K_{mix} to denote the number of wells in the mixture model.

For example, in `MixtureQuadraticVTheta` which represents the SQ3 variant of quadratic V_θ we use the following component mixture representation of the scalar potential $V_\theta = -\tau \log \sum_{k=1}^{K_{\text{mix}}} \pi_k e^{-E_k/\tau}$. In this formula we have K_{mix} components with weights π_k with energies E_k , combined by log-sum-exp.

How are banks assembled?

- **one bank (concat / joint):** `v_theta.banks` has length 1. That bank's d_{in} is d or $K_\xi d$. The context channels are no longer per bank, all channels feed this one mixture.
- **additive multi-context:** `n_ctx` banks, `banks[m]` reads only $\xi^{(m)}$ with $d_m = d$. Then $V_\theta = \sum_m V_{\text{bank}}^{(m)}$. For example, in many of our recent models we use 5 banks with 8 wells each abbrev. 5 heads $\times$ 8 wells.

One Bank (Concat / Joint)

Implemented via *Concat Adapters* in quadratic V_θ (SQ3) and early Gaussian implementations.

⁵ GMM – Gaussian Mixture Model, see [16] and [17] for additional info

Flatten $\xi \mapsto \mathbb{R}^{K_\xi d}$, one mixture, $d_{\text{in}} = K_\xi d$. No `V_theta.banks` or not used in the usual way.

Concat adapters are :

- `StructuredVThetaMultiXiAdapter` – used in the quadratic V_θ implementation `SQ3 MixtureQuadraticVTheta`.

Used on Multi- ξ SPLM / PARFLM / Fock-PARFLM v2.1 (A1–A5, Phase 4 OWT SQ3)

- `GaussianVThetaMultiXiAdapter` - used in isotropic `MixtureGaussianVTheta` and in `SARFGaussianVTheta`. Used on hybrid Gaussian–quadratic notebook and in Phase 5 SARF/Gaussian concat arms.

There is also a newer implementation which uses one *Joint Bank* (see [15])

- `JointContextAnisotropicGaussianVTheta` – used in `AnisotropicDepthConditionedGaussianVTheta`, code : [model_aniso_gaussian_vtheta.py](#).

The well centres are denoted with $\mu_k \in \mathbb{R}^d$. The diagonal precisions are denoted with $a_k \in \mathbb{R}_{>0}^d$ and represent the inverse variances per coordinate. The mixture weights are denoted with $w_k \in \mathbb{R}$.

More recently we have studied GMMs such as `MixtureGaussianVTheta` and `AnisotropicMixtureGaussianVTheta`. So, the inputs, parameters and outputs of those mixture models are:

- inputs : $\xi \in \mathbb{R}^{d_{\text{in}}}$, $h \in \mathbb{R}^d$
- parameters : the bank's `mu_proj`, `a_proj`, `w_proj`, (and `B_proj` if anisotropic)
- output: K_w wells $\{(\mu_k, P_k, w_k)\}_{k=1}^{K_w}$ and $V_{\text{bank}}(\xi, h) = -\sum_{k=1}^{K_w} w_k \exp\left(-\frac{1}{2}(h - \mu_k)^T P_k (h - \mu_k)\right)$

There are three maps, also denoted as *projections*, which are ordinary `nn.Linear` layers and their goal is to turn context only into Gaussian-mixture parameters:

Layer	Shape	Meaning after reshape / nonlinearity
<code>mu_proj</code>	$d_{\text{in}} \rightarrow K_w d$	Well centres $\mu_k \in \mathbb{R}^d$
<code>a_proj</code>	$d_{\text{in}} \rightarrow K_w d$	Diagonal precisions $a_k \in \mathbb{R}_{>0}^d$ (inverse variances per coordinate)
<code>w_proj</code>	$d_{\text{in}} \rightarrow K_w$	Mixture logits $\rightarrow$ softmax weights w_k

In the method `MixtureGaussianVTheta::_components` the following happens

$\mu = \text{reshape}(\text{mu_proj}(\xi)) \in \mathbb{R}^{K_w \times d}$

$a = \text{softplus}(\text{a_proj}(\xi)) + 10^{-4} \in \mathbb{R}^{K_w \times d}$ (then optional clamp)

$w = w_{\text{scale}} \cdot \text{softmax}(\text{w_proj}(\xi)) \in \mathbb{R}^{K_w}$

The potential for a single bank for isotropic Gaussian family is then:

$$V(\xi, h) = -\sum_{k=1}^{K_w} w_k(\xi) \exp\left(-\frac{1}{2} \sum_{i=1}^d a_{k,i}(\xi) (h_i - \mu_{k,i}(\xi))^2\right) \quad (\text{igp.1})$$

So,

`mu_proj` tells us where is the well k . It is *affine* in ξ because `mu_proj` is implemented as `nn.Linear`, that is: $\mu(\xi) = W_\mu \xi + b_\mu$.

`a_proj` tells us how sharp well k is along each axis. `softplus` keeps $a > 0$. With `init_log_precision = log d`, $a \approx \frac{1}{d}$ so $\sigma_{\text{eff}} \approx \sqrt{d}$, matching `LayerNorm` scale. `a_proj`, being `nn.Linear`, renders $a(\xi) = \text{softplus}(W_a \xi + b_a) + 10^{-4}$ which clearly is not affine transformation.

`w_proj` tells us which wells of the bank are **on** for this context ; softmax scales $\sum_k w_k = w_{\text{scale}}$ which is usually 1.

Anisotropic banks add an extra map : $B_proj: d_{in} \rightarrow K_w \cdot d \cdot r$, reshaped to $B_k \in \mathbb{R}^{d \times r}$ so the precision is $\text{diag}(a_k) + B_k B_k^T$. Here r is the rank of the off-diagonal precision factor for each well – it is given with constructor argument `rank`, typically is 4 on OWT anisotropic Gaussian Fock models. Recall, that each anisotropic Gaussian well k has a factor $B_k \in \mathbb{R}^{d \times r}$ obtained from `B_proj`. The precision (inverse covariance) P_k is the diagonal plus a rank- r PSD update:

$$P_k = \text{diag}(a_k) + B_k B_k^T \in \mathbb{R}^{d \times d}$$

$B_k B_k^T$ has rank of at most r , so the well is an ellipsoid that can tilt off the coordinate axes (correlations between hidden dimensions). `rank = 0` drops `B_proj` and $P_k = \text{diag}(a_k)$ and we recover the isotropic bank.

Contrast with MLP V_θ

The MLP first layer sees $\text{cat}(\xi, h)$. Gaussian projections see ξ only. h appears only in $(h - \mu)^T P (h - \mu)$. That is why centres can be read off as $\mu(\xi)$.

Where Does Flattening Happens?

The inner bank always assumes one context vector of length d_{in} . The multi- ξ context state is a tensor of shape (B, T, K_ξ, d) ,

$$\xi \in \mathbb{R}^{B \times T \times K_\xi \times d}.$$

The concat adapters `StructuredVThetaMultiXiAdapter` and `GaussianVThetaMultiXiAdapter` do not invent a new geometry for this tensor. They reshape the last two axes,

$$\xi_{\text{flat}} = \text{reshape}(\xi) \in \mathbb{R}^{K_\xi d},$$

and call the inner bank as $V = V_{\text{inner}}(\xi_{\text{flat}}, h)$. The hidden state h remains a separate argument. It is **not** concatenated into the well-parameter maps. Those maps see only the flattened context, with input width `xi_d` = $K_\xi d$:

`mu_proj` maps $K_\xi d$ to $K_w d$, `a_proj` maps $K_\xi d$ to $K_w d$, and `w_proj` maps $K_\xi d$ to K_w .

Every well parameter is therefore a dense linear mixture of **all** horizons. The model cannot dedicate an attractor to a single timescale: there is no well that “belongs” to the hundred-token exponential moving average, because every centre, precision, and mixture weight is allowed to depend on every channel. With the usual recipe $K_\xi = 4$ and $K_w = 8$ this produces eight attractors on a blurred $4d$ -dimensional summary, not thirty-two or forty horizon-specific centres.

A tiny example makes the wiring concrete. Take $K_\xi = 2$ channels, $K_w = 2$ wells, and $d = 3$. The concat adapter stacks $\xi^{(1)}, \xi^{(2)} \in \mathbb{R}^3$ into $\xi_{\text{flat}} \in \mathbb{R}^6$. Then `mu_proj` is an affine map $\mathbb{R}^6 \rightarrow \mathbb{R}^6$ whose six outputs are reshaped into two centres in $\mathbb{R}^3$. Each coordinate of μ_1 can depend on both the short-horizon and the long-horizon context. The additive alternative, developed earlier in this section and implemented by `MultiContextGaussianVTheta`, would instead give each channel its own bank: bank 1 is an affine map $\mathbb{R}^3 \rightarrow \mathbb{R}^6$ that sees only $\xi^{(1)}$, bank 2 is a twin map that sees only $\xi^{(2)}$, and the two banks together produce four centres, two per horizon.

Three things are **not** flattened, and confusing them with the adapter is a common source of misreading.

- The particle coordinate h never enters `mu_proj`, `a_proj`, or `w_proj`. Those three maps are functions of context only. The hidden state appears later, in the quadratic form or Gaussian bump that is evaluated at $h - \mu_k(\xi)$. That is why centres can be read off by calling `attractor_centres` without differentiating through h .
- The well index k is not a separate network. A single wide linear map writes a vector of length $K_w d$ (or K_w for the weights), and a reshape peels that vector into K_w blocks. Multi-context wiring duplicates the whole bank per channel; it does not allocate one MLP per well.

- The weight head w_proj emits K_w logits, not $K_w d$ numbers. Mixture weights are per well, not per hidden coordinate.

Flattening, in this architecture, is only an adapter that sets the input width of those three heads to $K_\xi d$ instead of d . The later OpenWebText Gaussian runs that quote “five heads times eight wells” use the other wiring: no flattening, five separate triples of (μ_proj , a_proj , w_proj), and a sum of five bank potentials. That design is parameter-neutral with the concat baseline **when both keep the same K_w per bank**: five maps $d \rightarrow 8d$ have the same number of weights as one map $5d \rightarrow 8d$, but they produce forty centres instead of eight, and they are block-diagonal across horizons rather than dense in the concatenated context. A dense map $5d \rightarrow 40d$ that tried to match the forty centres in a **single** concat bank would be five times heavier; that is not the concat baseline actually used.

Does the anisotropic Gaussian benefit beyond 4 Channels?

There is some benefit to giving an anisotropic Gaussian more than four K-EMA channels, but the returns diminish sharply and can become negative. They diminish faster than they do for the MLP V_θ . The reason is not that four is a magic number for language. It is that the additive multi-context Gaussian and the MLP spend extra channels in different ways.

In the additive design the force is a sum of per-bank forces,

$$f = -\nabla_h V_\theta = \sum_{m=1}^{n_{\text{ctx}}} f^{(m)}(\xi^{(m)}, h).$$

Each bank is expressive **within** its own horizon: centres $\mu_k^{(m)}(\xi^{(m)})$ and precisions $P_k^{(m)}(\xi^{(m)})$ are nonlinear functions of that channel. What the sum cannot represent is a conjunction across horizons. There is no attractor that fires only when the two-token channel says one thing **and** the two-hundred-token channel says another. Adding a channel therefore helps only if that horizon carries a useful **standalone** signal. It does not buy interaction terms. The MLP, by contrast, concatenates every $\xi^{(m)}$ with h and sends the joint vector through a shared nonlinearity, so gating, disagreement detectors, and other cross-horizon features are available in principle. Adjacent exponential moving averages are also strongly correlated. Channels with decays such as $\alpha = 0.99$ and $\alpha = 0.995$ are nested low-pass filters; every kernel passes the constant (direct-current) mode, so the effective number of independent directions in the bank is smaller than the nominal K_ξ . An extra Gaussian bank that sees a near-duplicate of an existing $\xi^{(m)}$ still adds a full set of well parameters and a full contribution to the force. The additive structure cannot rotate that redundancy away. The MLP’s first layer can down-weight or linearly combine correlated inputs at cost $d \cdot d_V$ per extra channel, without cloning a curvature machine.

That last point is specific to the anisotropic family. Every anisotropic bank has its own B_proj , producing a rank- r update $B_k B_k^T$ in the precision. Extra channels increase the independent off-diagonal curvature that the integrator has to confine (the “mechanism-A stiffness” of the CfC-BAOAB notes). The MLP has no per-channel ellipsoid at all. On OpenWebText Fock-PARFLM runs the K-EMA bank is therefore almost always left at four channels. For the MLP that choice makes the first layer a map $5d \rightarrow d_V$. For the additive Gaussian it is a compromise between covering a few genuinely different timescales and not cloning correlated force fields.

That is why proposals such as HiPPO-LegT or S4D, discussed in the multi-channel ξ notes, try to replace raw EMAs by more orthogonal filters rather than by simply taking K_ξ larger. Orthogonalising the bank is a change to how ξ is computed. It is not a substitute for giving V_θ a joint, rather than additive, reading of those channels — the subject of the next subsection and of [15].

Can we do something better than flattening?

Yes. Two alternatives are already in the code, and they attack different parts of the concat-adapter limitation. The first is the additive multi-context wiring already described: `MultiContextGaussianVTheta` and its depth-conditioned and anisotropic relatives. Flattening is abandoned; each channel owns a bank of K_w wells; the potential is the sum of the bank scalars. That restores horizon-specific attractors at the same parameter count as a

concat bank with the same K_w . It does **not** restore cross-horizon conjunctions, which is why extra channels still saturate, as the previous subsection argued.

The second is a **joint** bank whose well parameters are functions of the concatenated context, implemented as `JointContextAnisotropicGaussianVTheta` and selected on

`AnisotropicDepthConditionedGaussianVTheta` by `coupling='joint'`. Analyticity of the force requires only that μ_k, a_k, B_k and w_k depend on ξ and not on h . They are allowed to be arbitrary joint functions of all channels. A single projection $n_{\text{ctx}}d \rightarrow K_w d$ is the simplest such function: every well can see every horizon, so additive separability is broken, while the force, the Hessian in h , and the CfC-BAOAB harmonic split remain closed form. To match the forty attractors of a 5×8 additive stack one sets $K_w = 40$ on that one bank, accepting a denser map. Companion note [15] (`Analytic_Multi_Channel_Integration_in_Structured_Vtheta.md`) develops the rest of the ladder — conjunctive gates, product-of-experts fusion of Gaussian factors, bilinear cross-features — and the curvature caveat: fusing channels concentrates stiffness rather than cloning it. The joint projection is the drop-in prototype; the other rungs remain design options. Nothing in that note changes the K-EMA computation of ξ itself. Geodesic structure still lives in how the integrator walks $V_\theta(h; \xi)$, not in how the prefix is averaged.

Depth conditioning and context banks in structured V theta

The structured Gaussian used on OpenWebText Fock-PARFLM is not a concat adapter. It is a **depth-conditioned context-bank** module: several well banks, one per K-EMA channel, sharing their parameters across Verlet layers except for a small learned shift of the context. The isotropic form is

`DepthConditionedMultiContextGaussianVTheta` in `model_gaussian_vtheta.py`. The anisotropic form is `AnisotropicDepthConditionedGaussianVTheta` in `model_aniso_gaussian_vtheta.py`. Both read the same multi-channel tensor $\xi \in \mathbb{R}^{B \times T \times n_{\text{ctx}} \times d}$ and the same particle coordinate $h \in \mathbb{R}^d$.

Context banks

Write n_{ctx} for the number of banks and K_w for the number of wells inside each bank. The OpenWebText default is $n_{\text{ctx}} = 5$ and $K_w = 8$, hence forty Gaussian terms. Bank m sees only its own channel $\xi^{(m)} \in \mathbb{R}^d$. Its three maps are therefore narrow,

Bank m therefore has `mu_proj` mapping d to $K_w d$, `a_proj` mapping d to $K_w d$, and `w_proj` mapping d to K_w , and the total potential is the sum already written as (`mcgvt.3`),

$$V_\theta(\xi, h) = \sum_{m=1}^{n_{\text{ctx}}} V^{(m)}(\xi^{(m)}, h).$$

Each centre is a function of a single timescale. The linear mass of the centre maps is the same as that of a concat adapter with the same K_w , but the forty centres are not forty linear mixtures of a blurred $5d$ vector; they are eight centres on channel 1, eight on channel 2, and so on. Depth conditioning, below, does not add a fourth projection. It only shifts the input that these three maps already see.

Depth codes as a per-layer shift

Fully untying V_θ across the L integration layers would copy the entire bank stack L times. At $d = 384$, $K_w = 8$, $n_{\text{ctx}} = 5$ and $L = 16$ that is tens of millions of extra parameters, almost all of them in the dense $d \rightarrow K_w d$ maps. The depth-conditioned module keeps **one** shared stack and introduces a learned table

$$e \in \mathbb{R}^{L \times n_{\text{ctx}} \times d},$$

the `depth_code` parameter, initialised from $\mathcal{N}(0, 0.02^2)$. Before the bank projections at layer ℓ one replaces

$$\xi^{(m)} \text{ arrow } \xi^{(m)} + e_{\ell, m}.$$

The code is constant with respect to h , so the force remains $f = -\nabla_h V_\theta$ of a scalar; only the conditioning has changed. The extra cost is $L n_{\text{ctx}} d$ parameters (about thirty thousand at the numbers above), not a second copy of the banks. The well **shapes** stay tied across depth; what varies with ℓ is which view of ξ those shapes are evaluated at. The layer index is not an extra argument of forward. The model's per-layer step writes it into `_active_layer` (`install_depth_routing` on `FockMultiXiPARFLM`), including during gradient-checkpoint recomputation, and the module reads that pointer when it shifts ξ .

Where the forty wells sit before training

The forty wells are not a stored lookup table of points. Each centre is an affine image of its channel,

$$\mu_{m,k}(\xi^{(m)}) = \text{reshape}(W_m \xi^{(m)} + b_m)_k \in \mathbb{R}^d.$$

There is no buffer of frozen PMI anchors — that construction belongs to `SARFGaussianVTheta`, a different class.

The biases b_m are initialised to zero in both the isotropic and the anisotropic families. Consequently, if the context vanished, every centre would sit at the origin: a forty-fold degeneracy. On a real first batch, ξ is a K-EMA of embeddings drawn from $\mathcal{N}(0, 0.02^2)$ plus positional codes, so $\mu = W\xi$ is only a tiny random scatter about zero, not a lattice, a simplex, or a set of token prototypes.

The two families differ in how small that scatter is. Isotropic `mu_proj` uses Gaussian weights of standard deviation 0.02. Anisotropic `mu_proj` uses Xavier uniform on `Linear(d, K_w d)`, whose bound is

$\sqrt{6/(d + K_w d)} = \sqrt{2/(3d)}$ when $K_w = 8$, about 0.042 at $d = 384$. In both cases ξ itself is $O(1/\sqrt{d})$ after LayerNorm-scale embeddings, so all forty centres start on top of each other near the origin. They do not tile the LayerNorm sphere.

The mixture weights start uniform. On the anisotropic bank that is exact: `w_proj` is initialised with zero weight and zero bias, so the softmax logits are identically zero and each well in a bank carries mass $1/K_w = 1/8$. Precisions are initialised from `init_log_precision = -log d`, which gives an effective width $\sigma_{\text{eff}} \approx \sqrt{d}$. On a LayerNorm sphere of radius $\sqrt{d}$ that is a stack of **wide, overlapping bowls** around the origin, not forty separated basins. Saying that “forty attractors exist from step 0” is a statement about the **parameterisation** — there are forty Gaussian terms in the formula — not about geometry. Distinct centres have to be learned as the maps W_m and the depth codes move.

They are in particular **not** initialised to random points on the LayerNorm sphere, to vocabulary or embedding prototypes, to a k -means of hidden states, or to SARF PMI-peak anchors. A direct check is to call `V_theta.attractor_centres(xis)` on the first training batch, before `optimizer.step()`. One should see forty vectors with Euclidean norms much smaller than 1 and pairwise distances much smaller than $\sqrt{d}$, all clustered at the origin.

Module `splm_oracle_fit.py`

Function `fit_scalars`

It finds $\arg \min \|Y - \alpha V - \beta(-G)\|^2$ that it fits $Y \approx \alpha V + \beta(-G)$ per layer.

Here:

Y – the regression target : per-step displacement $\Delta h_\ell = h_{\ell+1} - h_\ell$. Shape: $(L, T, d)^6$, i.e.

```
def fit_scalars(V: np.ndarray, G: np.ndarray, Y: np.ndarray, LAY: np.ndarray):
    """Per-layer least-squares: [alpha_l, beta_l] = argmin || Y - alpha V - beta*(-G) ||^2.
    V: np.ndarray - the velocity proxy
    G: np.ndarray - the potential gradient
    Y: np.ndarray - the regression target
    LAY: np.ndarray - layer indices
    """
    alpha = {}; beta = {}
    Y_pred = np.zeros_like(Y)
    for ell in np.unique(LAY):
```

⁶ The shape of the *live forward-pass* tensor h is (B, T, d) but `fit_scalars` never sees the live tensor. Instead, `fit_scalars` operates on the numpy arrays produced by `extract_oracle_tuples` which changes the shape of h . For details see the Appendix subsection on `extract_oracle_tuples`.

```

m = LAY == ell
v = V[m].reshape(-1); g = (-G[m]).reshape(-1); y = Y[m].reshape(-1)
A = np.stack([v, g], axis=1) # (N*d, 2)
coef, *_ = np.linalg.lstsq(A, y, rcond=None)
a, b = coef
alpha[int(ell)] = float(a); beta[int(ell)] = float(b)
Y_pred[m] = a * V[m] + b * (-G[m])
return Y_pred, alpha, beta

```

Function `extract_oracle_tuples`

What does `extract_oracle_tuples` do

It takes a model, a raw string, and a device; returns `(H, GV, XI)`. The shapes of these are:

`H: (L+1, T, d), GV: (L, T, d), XI: (T, d)`.

`extract_oracle_tuples` replays the system dynamics via the damped semi-implicit Euler form:

$$h_{\ell+1} = h_{\ell} + \Delta t \cdot v_{\ell+1}; v_{\ell+1} = (v_{\ell} + \Delta t \cdot f_{\ell}/m)/(1 + \Delta t \cdot \gamma); f_{\ell} = -\nabla_h V_{\theta}(\xi, h_{\ell})$$

The lines

```

max_len = model.cfg.max_len
ids = tokenize(sentence, max_len)
x = torch.from_numpy(ids).unsqueeze(0).to(device)

```

tokenize the sentence to ids, then `unsqueeze(0)` adds a batch dimension of 1 so `x` has shape `(1,T)`.

This is the `B = 1` that later gets dropped by the `[0]` indexing, which is why the outputs are `(L+1,T,d)` instead of `(B,T,d)` which was the shape of the raw `h` tensor in the forward pass.

The lines below define two capability flags that drive all the branching: whether to re-pool ξ per step, and whether to apply the LayerNorm-after-step projection:

```

is_sarf = isinstance(model, ScalarPotentialLMSARFMass)
has_ln = hasattr(model, '_project') and getattr(cfg, 'ln_after_step', False)

```

Next gradient track is enabled via the line

```

with torch.enable_grad():

```

On the code shown below we have two branches: a) the SARF branch - it embeds, computes a possibly per-token mass `m_b`, and optionally LN-projects the initial state. b) Base branch: `_embed_and_pool` returns both the embedding *and* the frozen ξ computed once (the fixed- ξ semantics), and mass is the single global scalar `model.m`. In both, `h` is now $h^{(0)}$ - the decontextualized initial embedding (or its LN projection)

```

if is_sarf:
    emb = model._embed(x)
    m_b = model.compute_mass(x, emb)
    gamma = model.gamma
    h = model._project(emb) if has_ln else emb
else:
    emb, xi_fixed = model._embed_and_pool(x)
    m_b = model.m
    gamma = model.gamma

```

```
h = emb
```

And here is the full code:

```
def extract_oracle_tuples(model, sentence: str, device: str) \
    -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """For a single sentence, re-run SPLM and return
    H   : (L+1, T, d) raw hidden states h_l
    GV  : (L, T, d) grad_h V_theta(xi, h_l) for l = 0..L-1
    XI  : (T, d) the constant context vector

    The dynamics equation is

    h_{l+1} = h_l + dt * v_{l+1}
    v_{l+1} = (v_l + dt*f_l/m) / (1 + dt*gamma), f_l = -grad_h V_theta(xi, h_l)

    so Delta h_l := h_{l+1} - h_l depends on (v_l, grad_V at h_l).

    Handles both the base ScalarPotentialLM (fixed xi from _embed_and_pool)
    and ScalarPotentialLMSARFMass[LN] variants (per-step xi via
    causal_cumulative_mean, per-token mass, optional LN projection).
    """
    from model_sarf_mass import ScalarPotentialLMSARFMass, causal_cumulative_mean

    max_len = model.cfg.max_len
    ids = tokenize(sentence, max_len)
    x = torch.from_numpy(ids).unsqueeze(0).to(device)
    cfg = model.cfg
    is_sarf = isinstance(model, ScalarPotentialLMSARFMass)
    has_ln = hasattr(model, '_project') and getattr(cfg, 'ln_after_step', False)

    with torch.enable_grad():
        if is_sarf:
            emb = model._embed(x)
            m_b = model.compute_mass(x, emb)
            gamma = model.gamma
            h = model._project(emb) if has_ln else emb
        else:
            emb, xi_fixed = model._embed_and_pool(x)
            m_b = model.m
            gamma = model.gamma
            h = emb

        v = torch.zeros_like(h)
        dt = cfg.dt

        H = [h.detach().cpu().numpy()[0]]
        GV = []
        xi_out = None
        for _ in range(cfg.L):
            if is_sarf:
                xi_input = h.detach() if getattr(cfg, 'causal_force', True) else h
                xi_now = causal_cumulative_mean(xi_input)
            else:
                xi_now = xi_fixed
```



```

h_in = h.detach().requires_grad_(True)
V_out = model.V_theta(xi_now, h_in).sum()
grad_V, = torch.autograd.grad(V_out, h_in)
GV.append(grad_V.detach().cpu().numpy()[0])
f = -grad_V
v = (v + dt * f / m_b) / (1.0 + dt * gamma)
h_new = h_in + dt * v
if has_ln:
    h_new = model._project(h_new)
h = h_new
H.append(h.detach().cpu().numpy()[0])
if xi_out is None:
    xi_out = xi_now

xi_np = xi_out.detach().cpu().numpy()[0].astype(np.float32)
return (np.stack(H, axis=0).astype(np.float32),
        np.stack(GV, axis=0).astype(np.float32),
        xi_np)

```

Module `model.py`

The SPLM Config : class `SPLMConfig`

```

@dataclass
class SPLMConfig:
    vocab_size: int = 50257
    d: int = 128
    max_len: int = 512
    v_hidden: int = 512
    v_depth: int = 3          # number of hidden layers in V_theta MLP
    L: int = 8                # integration steps (shared-weight "depth")
    dt: float = 1.0
    init_m: float = 1.0
    init_gamma: float = 1.0
    learn_mgamma: bool = True

```

Class `ScalarPotential`

Note: the class `ScalarPotential` in module `model.py` is identical to the same class in the modules `model_ln.py` and `model_sarf_mass.py`.

```

class ScalarPotential(nn.Module):
    """MLP (xi, h) in R^(2d) -> scalar energy."""

    def __init__(self, d: int, hidden: int, depth: int):
        super().__init__()
        layers: List[nn.Module] = [nn.Linear(2 * d, hidden), nn.GELU()]
        for _ in range(depth - 1):
            layers += [nn.Linear(hidden, hidden), nn.GELU()]
        layers += [nn.Linear(hidden, 1)]
        self.net = nn.Sequential(*layers)
        # Gentle init so gradients are small at start -- integrator stays stable.
        for m in self.net.modules():
            if isinstance(m, nn.Linear):

```

```

        nn.init.normal_(m.weight, std=0.02)
        if m.bias is not None:
            nn.init.zeros_(m.bias)
    # Final layer even smaller -- the initial force should be tiny.
    last = [m for m in self.net.modules() if isinstance(m, nn.Linear)][-1]
    nn.init.normal_(last.weight, std=0.002)
    nn.init.zeros_(last.bias)

def forward(self, xi: torch.Tensor, h: torch.Tensor) -> torch.Tensor:
    """xi: (... , d), h: (... , d) -> V: (... , 1)"""
    return self.net(torch.cat([xi, h], dim=-1))

```

Class ScalarPotentialLM

```

class ScalarPotentialLM(nn.Module):
    def __init__(self, cfg: SPLMConfig):
        super().__init__()
        self.cfg = cfg
        self.E = nn.Embedding(cfg.vocab_size, cfg.d)
        self.P = nn.Parameter(torch.zeros(cfg.max_len, cfg.d))
        nn.init.normal_(self.E.weight, std=0.02)
        nn.init.normal_(self.P, std=0.01)

        self.V_theta = ScalarPotential(cfg.d, cfg.v_hidden, cfg.v_depth)

        # Positive-definite mass / damping via softplus.
        # raw values chosen so F.softplus(raw) ~ init_m / init_gamma.
        def _raw_from_positive(y: float) -> float:
            import math
            return math.log(math.exp(1(max(y, 1e-3)))

        self.raw_m = nn.Parameter(torch.tensor(_raw_from_positive(cfg.init_m)),
                                   requires_grad=cfg.learn_mgamma)
        self.raw_gamma = nn.Parameter(torch.tensor(_raw_from_positive(cfg.init_gamma)),
                                       requires_grad=cfg.learn_mgamma)

    @property
    def m(self) -> torch.Tensor:
        return F.softplus(self.raw_m) + 1e-3

    @property
    def gamma(self) -> torch.Tensor:
        return F.softplus(self.raw_gamma)

    # -----
    def _embed_and_pool(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
        """x: (B, T) token ids -> (emb, xi)"""
        B, T = x.shape
        pos = self.P[:T].unsqueeze(0) # (1, T, d)
        emb = self.E(x) + pos # (B, T, d)
        cumsum = emb.cumsum(dim=1) # (B, T, d)
        denom = torch.arange(1, T + 1, device=x.device,
                              dtype=emb.dtype).view(1, T, 1)
        xi = cumsum / denom # causal cumulative mean
        return emb, xi

```

```

# -----
def integrate(
    self,
    emb: torch.Tensor,  # (B, T, d) initial h_0
    xi: torch.Tensor,   # (B, T, d) context (held fixed along integration)
    return_trajectory: bool = False,
) -> Tuple[torch.Tensor, Optional[List[torch.Tensor]]]:
    """Damped Lagrangian flow starting from h_0 = emb, v_0 = 0.

    Returns h_L (and optionally the full per-step trajectory
    [h_0, h_1, ..., h_L] as a list of detached tensors for analysis).
    """
    cfg = self.cfg
    h = emb
    v = torch.zeros_like(h)
    m, gamma, dt = self.m, self.gamma, cfg.dt

    traj: Optional[List[torch.Tensor]] = None
    if return_trajectory:
        traj = [h.detach().cpu()]

    for _ in range(cfg.L):
        # Force f = -grad_h V_theta(xi, h), differentiable through V_theta.
        h_in = h
        if not h_in.requires_grad:
            h_in = h_in.requires_grad_(True)
        V = self.V_theta(xi, h_in).sum()
        grad_V, = torch.autograd.grad(
            V, h_in, create_graph=self.training, retain_graph=True,
        )
        f = -grad_V
        v = (v + dt * f / m) / (1.0 + dt * gamma)
        h = h_in + dt * v
        if return_trajectory:
            assert traj is not None
            traj.append(h.detach().cpu())

    return h, traj

# -----
def forward(
    self,
    x: torch.Tensor,
    targets: Optional[torch.Tensor] = None,
    return_trajectory: bool = False,
):
    """x: (B, T) token ids, targets: (B, T) next-token ids (or None).

    Returns (logits, loss, traj?). traj is a list of (B, T, d)
    tensors on CPU only when return_trajectory=True.
    """
    emb, xi = self._embed_and_pool(x)
    h_L, traj = self.integrate(emb, xi, return_trajectory=return_trajectory)

    logits = h_L @ self.E.weight.T  # (B, T, V)

```

```

    loss = None
    if targets is not None:
        loss = F.cross_entropy(
            logits.reshape(-1, self.cfg.vocab_size),
            targets.reshape(-1),
        )
    if return_trajectory:
        return logits, loss, traj
    return logits, loss

# -----
@torch.no_grad()
def generate(self, x: torch.Tensor, max_new_tokens: int,
            temperature: float = 1.0, top_k: Optional[int] = None) -> torch.Tensor:
    """Autoregressive greedy / sampled generation. x: (1, T0)."""
    self.eval()
    for _ in range(max_new_tokens):
        x_cond = x[:, -self.cfg.max_len:]
        logits, _ = self.forward(x_cond)
        logits = logits[:, -1, :] / max(temperature, 1e-6)
        if top_k is not None:
            v, _ = torch.topk(logits, top_k)
            logits[logits < v[:, [-1]]] = -float("inf")
        probs = F.softmax(logits, dim=-1)
        nxt = torch.multinomial(probs, num_samples=1)
        x = torch.cat([x, nxt], dim=1)
    return x

# -----
def num_params(self) -> int:
    return sum(p.numel() for p in self.parameters() if p.requires_grad)

```

Putting it together: the reference forward pass

The three pieces above compose into a single forward pass. `_embed_and_pool` builds the initial state $h_0 = E[x] + P$ and, in the same call, the context x_i as the causal cumulative mean of those embeddings — computed once and then held fixed for the entire integration. `integrate` runs the L shared damped Euler-Lagrange steps from h_0 with $v_0 = 0$, at each step taking the force $f = -\text{grad}_h V_{\text{theta}}(x_i, h)$ and updating v then h . `forward` reads out tied-embedding logits $h_L @ E^T$ and, when targets are supplied, the next-token cross-entropy. `generate` and `num_params` are utilities — autoregressive sampling and a trainable-parameter count — and introduce no new dynamics.

Two design choices distinguish this reference model from its descendants and are worth stating explicitly. First, the context is fixed- x_i : x_i is derived once from the initial embeddings and never recomputed, so every layer differentiates the same two-argument potential against the same conditioning slice. This is the simplest way to honour the asymmetric role of x_i (frozen conditioner) versus h (differentiation variable), and it makes the per-token force trivially curl-free. Second, the mass is a single global scalar `self.m` (softplus of one learned parameter) shared by every token and position; there is no LayerNorm, and the domain of h is unconstrained.

These choices place `model.py` at the root of the model family. `model_sarf_mass.py` relaxes the first by re-pooling x_i at every step from $h.detach()$ (the SARF, causal-force-correct dynamics) and the second by making the mass per-token (`global`, `embed_head`, or `logfreq`). `model_ln.py` then adds a LayerNorm-after-step projection on top of the SARF model, compactifying the domain. The integration kernel — damped semi-implicit Euler with a force read

from a single shared scalar potential — is identical across all three; only the conditioning, the mass, and the domain change.

Module `model_sarf_mass.py`

The SARF Config- class `SPLMSARFMassConfig`

```
@dataclass
class SPLMSARFMassConfig:
    vocab_size: int = 50257
    d: int = 128
    max_len: int = 512
    v_hidden: int = 512
    v_depth: int = 3
    L: int = 8
    dt: float = 1.0
    init_m: float = 1.0
    init_gamma: float = 1.0
    learn_mgamma: bool = True

    mass_mode: str = "global"
    logfreq_init_alpha: float = 0.0
    logfreq_path: Optional[str] = None

    fixed_gamma: Optional[float] = None

    # When True (default), compute  $\xi$  from h.detach() inside the integration
    # loop. This severs the autograd path from  $\xi$  back to h, eliminating an
    # anti-causal leak in the conservative force where  $\partial V[t'] / \partial h[t] \neq 0$  for
    #  $t' > t$  (because  $\xi[t']$  is a causal weighted average that includes h[t]).
    # The fix restores the physics-correct Euler-Lagrange equation
    #  $m \cdot \ddot{h}_t = -\partial V(\xi_t, h_t) / \partial h_t$ 
    # as the per-token dynamics. See docs/Causal_Leak_in_SPLM_Integrate_Bug_and_Fix.md
    # for the full bug-and-fix writeup. Set causal_force=False ONLY to
    # bit-exactly reproduce pre-fix experiments (e.g. E9 forensics).
    causal_force: bool = True
```

Function `causal_cumulative_mean`

This function computes a running (prefix) mean over the sequence dimension, in a causal way — each position gets the average of itself and everything before it, with no leakage from future positions.

```
def causal_cumulative_mean(h: torch.Tensor) -> torch.Tensor:
    T = h.shape[1]
    cumsum = h.cumsum(dim=1)
    denom = torch.arange(1, T + 1, device=h.device, dtype=h.dtype).view(1, T, 1)
    return cumsum / denom
```

Class `ScalarPotentialLLMSARFMass`

Note on method `compute_mass`:

`compute_mass` produces the semantic field m for each token, shaped so it broadcasts over (B, T, d) , and dispatches on `cfg.mass_mode` between three strategies of increasing structure.

“`global`” is the trivial case : it returns `self.m_global` directly, a single learned mass shared by every token and position. No dependence on identity or context – just one scalar / 1-element tensor the optimizer tunes. This is our baseline mass.

“`embed_head`” makes mass a learned function of token identity. It re-embeds the ids with `self.E(x)` to get (B, T, d) embeddings, runs them through `self.mass_head` (a small projection down to a scalar-per-token), then maps that raw output to a strictly positive mass via `F.softplus(raw + self.raw_m_bias) + 1e-3`. The `raw_m_bias` shifts the operating point of the softplus, so the head doesn’t have to learn the baseline magnitude itself, and the `+ 1e-3` floor guarantees m never collapses toward zero. So here different tokens can carry systematically different mass, learned end-to-end.

“`logfreq`” ties mass to token rarity instead of letting it be free. `self.logfreq_surprisal[x]` looks up precomputed surprisal (essentially $-\log p(\text{token})$) per id, so rare tokens score high and frequent tokens low.

`alpha = F.softplus(self.raw_logfreq_alpha)`⁷ is a learned, strictly-positive gain on that surprisal.

```
def compute_mass(self, x: torch.Tensor, emb: torch.Tensor) -> torch.Tensor:
    """Return m as a tensor broadcastable over (B, T, d).

    x:      (B, T)          token ids
    emb:    (B, T, d)       first-layer hidden state E[x] + P
    """
    cfg = self.cfg
    if cfg.mass_mode == "global":
        return self.m_global

    if cfg.mass_mode == "embed_head":
        raw = self.mass_head(self.E(x))
        return (F.softplus(raw + self.raw_m_bias) + 1e-3)

    if cfg.mass_mode == "logfreq":
        surprisal = self.logfreq_surprisal[x]
        alpha = F.softplus(self.raw_logfreq_alpha)
        scaled = alpha * surprisal.unsqueeze(-1)
        return F.softplus(self.raw_m_bias + scaled) + 1e-3

    raise RuntimeError("unreachable")
```

Full code of this class below:

```
class ScalarPotentialLMSARFMass(nn.Module):
```

⁷ In `torch.nn.functional.softplus` $\text{Softplus}(x) = \frac{1}{\beta} \cdot \log(1 + \exp(\beta \cdot x))$; for numerical stability $\text{Softplus}(x)$ reverts to linear function when $\beta \cdot x > \text{threshold}$

```

"""SARF-faithful SPLM with pluggable per-token mass."""

def __init__(self, cfg: SPLMSARFMassConfig):
    super().__init__()
    self.cfg = cfg
    self.E = nn.Embedding(cfg.vocab_size, cfg.d)
    self.P = nn.Parameter(torch.zeros(cfg.max_len, cfg.d))
    nn.init.normal_(self.E.weight, std=0.02)
    nn.init.normal_(self.P, std=0.01)

    self.V_theta = ScalarPotential(cfg.d, cfg.v_hidden, cfg.v_depth)

    self.raw_m_bias = nn.Parameter(
        torch.tensor(_raw_from_positive(cfg.init_m)),
        requires_grad=cfg.learn_mgamma,
    )
    if cfg.fixed_gamma is not None:
        self.raw_gamma = nn.Parameter(
            torch.tensor(0.0),
            requires_grad=False,
        )
        self._gamma_value: Optional[float] = float(cfg.fixed_gamma)
    else:
        self.raw_gamma = nn.Parameter(
            torch.tensor(_raw_from_positive(cfg.init_gamma)),
            requires_grad=cfg.learn_mgamma,
        )
        self._gamma_value = None

    if cfg.mass_mode == "global":
        pass
    elif cfg.mass_mode == "embed_head":
        self.mass_head = nn.Linear(cfg.d, 1, bias=True)
        nn.init.zeros_(self.mass_head.weight)
        nn.init.zeros_(self.mass_head.bias)
    elif cfg.mass_mode == "logfreq":
        if cfg.logfreq_path is None:
            raise ValueError(
                "mass_mode='logfreq' requires cfg.logfreq_path (a .numpy file "
                "with one surprisal value per vocabulary id)."
            )
        surprisal = torch.from_numpy(
            _load_numpy(cfg.logfreq_path)
        ).float()
        if surprisal.numel() != cfg.vocab_size:
            raise ValueError(
                f"logfreq vector length {surprisal.numel()} != "
                f"vocab_size {cfg.vocab_size}"
            )
        self.register_buffer("logfreq_surprisal", surprisal)
        self.raw_logfreq_alpha = nn.Parameter(
            torch.tensor(_raw_from_positive(max(cfg.logfreq_init_alpha, 1e-3))),
            requires_grad=True,
        )
    else:
        raise ValueError(f"unknown mass_mode: {cfg.mass_mode!r}")

```

```

@property
def gamma(self) -> torch.Tensor:
    if self._gamma_value is not None:
        return torch.full(
            (), self._gamma_value,
            device=self.raw_gamma.device, dtype=self.raw_gamma.dtype,
        )
    return F.softplus(self.raw_gamma)

@property
def m_global(self) -> torch.Tensor:
    return F.softplus(self.raw_m_bias) + 1e-3

def compute_mass(self, x: torch.Tensor, emb: torch.Tensor) -> torch.Tensor:
    """Return m as a tensor broadcastable over (B, T, d).

    x:      (B, T)          token ids
    emb:    (B, T, d)       first-layer hidden state E[x] + P
    """
    cfg = self.cfg
    if cfg.mass_mode == "global":
        return self.m_global

    if cfg.mass_mode == "embed_head":
        raw = self.mass_head(self.E(x))
        return (F.softplus(raw + self.raw_m_bias) + 1e-3)

    if cfg.mass_mode == "logfreq":
        surprisal = self.logfreq_surprisal[x]
        alpha = F.softplus(self.raw_logfreq_alpha)
        scaled = alpha * surprisal.unsqueeze(-1)
        return F.softplus(self.raw_m_bias + scaled) + 1e-3

    raise RuntimeError("unreachable")

def _embed(self, x: torch.Tensor) -> torch.Tensor:
    B, T = x.shape
    pos = self.P[:T].unsqueeze(0)
    return self.E(x) + pos

def integrate(
    self,
    x: torch.Tensor,
    emb: torch.Tensor,
    return_trajectory: bool = False,
    return_xi_trajectory: bool = False,
) -> Tuple[torch.Tensor, Optional[List[torch.Tensor]], Optional[List[torch.Tensor]]]:
    """SARF-faithful damped EL with per-token mass.

    x:      (B, T)
    emb:    (B, T, d)   first-layer hidden state (with position embed)
    """
    cfg = self.cfg
    h = emb
    v = torch.zeros_like(h)

```



```

gamma, dt = self.gamma, cfg.dt

m = self.compute_mass(x, emb)
if m.dim() == 0:
    m_b = m
else:
    m_b = m

traj_h: Optional[List[torch.Tensor]] = None
traj_xi: Optional[List[torch.Tensor]] = None
if return_trajectory:
    traj_h = [h.detach().cpu()]
if return_xi_trajectory:
    traj_xi = []

for _ in range(cfg.L):
    xi_input = h.detach() if cfg.causal_force else h
    xi_now = causal_cumulative_mean(xi_input)
    if return_xi_trajectory:
        assert traj_xi is not None
        traj_xi.append(xi_now.detach().cpu())

    h_in = h
    if not h_in.requires_grad:
        h_in = h_in.requires_grad_(True)
    V = self.V_theta(xi_now, h_in).sum()
    grad_V, = torch.autograd.grad(
        V, h_in, create_graph=self.training, retain_graph=True,
    )
    f = -grad_V
    v = (v + dt * f / m_b) / (1.0 + dt * gamma)
    h = h_in + dt * v
    if return_trajectory:
        assert traj_h is not None
        traj_h.append(h.detach().cpu())

return h, traj_h, traj_xi

def forward(
    self,
    x: torch.Tensor,
    targets: Optional[torch.Tensor] = None,
    return_trajectory: bool = False,
    return_xi_trajectory: bool = False,
):
    emb = self._embed(x)
    h_L, traj_h, traj_xi = self.integrate(
        x, emb,
        return_trajectory=return_trajectory,
        return_xi_trajectory=return_xi_trajectory,
    )
    logits = h_L @ self.E.weight.T

    loss = None
    if targets is not None:
        loss = F.cross_entropy(

```

```

        logits.reshape(-1, self.cfg.vocab_size),
        targets.reshape(-1),
    )
    out = [logits, loss]
    if return_trajectory:
        out.append(traj_h)
    if return_xi_trajectory:
        out.append(traj_xi)
    return tuple(out) if len(out) > 2 else (out[0], out[1])

@torch.no_grad()
def generate(self, x: torch.Tensor, max_new_tokens: int,
            temperature: float = 1.0, top_k: Optional[int] = None) -> torch.Tensor:
    self.eval()
    for _ in range(max_new_tokens):
        x_cond = x[:, -self.cfg.max_len:]
        logits, _ = self.forward(x_cond)
        logits = logits[:, -1, :] / max(temperature, 1e-6)
        if top_k is not None:
            v, _ = torch.topk(logits, top_k)
            logits[logits < v[:, [-1]]] = -float("inf")
        probs = F.softmax(logits, dim=-1)
        nxt = torch.multinomial(probs, num_samples=1)
        x = torch.cat([x, nxt], dim=1)
    return x

def num_params(self) -> int:
    return sum(p.numel() for p in self.parameters() if p.requires_grad)

@torch.no_grad()
def mass_stats(self, x: torch.Tensor) -> dict:
    """Diagnostic summary of the per-token mass on a batch of tokens."""
    emb = self._embed(x)
    m = self.compute_mass(x, emb)
    if m.dim() == 0:
        return {"mean": float(m.item()), "std": 0.0,
                "min": float(m.item()), "max": float(m.item())}
    m_flat = m.reshape(-1)
    return {
        "mean": float(m_flat.mean().item()),
        "std": float(m_flat.std().item()),
        "min": float(m_flat.min().item()),
        "max": float(m_flat.max().item()),
    }

```

Module model_ln.py

The Config Class: SPLMSARFMassLNConfig

```

@dataclass
class SPLMSARFMassLNConfig(SPLMSARFMassConfig):
    """Extends SPLMSARFMassConfig with LayerNorm-after-step switches."""
    ln_eps: float = 1e-5
    ln_after_step: bool = True
    ln_affine: bool = False

```

Class ScalarPotentialLMSARFMassLN

model_ln.py makes exactly one change to the SARF-faithful SPLM: after every damped integration step it projects the hidden state back onto the unit-LayerNorm shell. V_{θ} , the mass machinery, the causal xi re-pooling, and the integrator are all inherited unchanged from ScalarPotentialLMSARFMass. The config SPLMSARFMassLNConfig (above) simply adds three switches — ln_eps, ln_after_step, and ln_affine — to the SARF config it subclasses. The class subclasses the SARF model and overrides only __init__ (to optionally allocate an affine nn.LayerNorm), adds the projection method _project, and overrides integrate to apply that projection. When ln_affine is False (the default) the projection is parameter-free — $F.\text{layer_norm}(h, (d,), \text{eps})$ removes each token's mean and scales it to unit standard deviation with no learned gain or bias. When ln_affine is True a standard nn.LayerNorm(d) with learnable affine parameters is used instead.

```
class ScalarPotentialLMSARFMassLN(ScalarPotentialLMSARFMass):
    """SARF-faithful SPLM with mandatory LayerNorm after every damped step."""

    def __init__(self, cfg: SPLMSARFMassLNConfig):
        super().__init__(cfg)
        self.cfg: SPLMSARFMassLNConfig = cfg
        if cfg.ln_affine:
            self.post_ln = nn.LayerNorm(cfg.d, eps=cfg.ln_eps)
        else:
            self.post_ln = None

    def _project(self, h: torch.Tensor) -> torch.Tensor:
        if self.post_ln is not None:
            return self.post_ln(h)
        return F.layer_norm(h, (self.cfg.d, ), eps=self.cfg.ln_eps)

    def integrate(self, x, emb, return_trajectory=False,
                  return_xi_trajectory=False):
        cfg = self.cfg
        h = self._project(emb) if cfg.ln_after_step else emb
        v = torch.zeros_like(h)
        gamma, dt = self.gamma, cfg.dt
        m_b = self.compute_mass(x, emb)

        traj_h = [h.detach().cpu()] if return_trajectory else None
        traj_xi = [] if return_xi_trajectory else None

        for _ in range(cfg.L):
            xi_input = h.detach() if cfg.causal_force else h
            xi_now = causal_cumulative_mean(xi_input)
            if return_xi_trajectory:
                traj_xi.append(xi_now.detach().cpu())

            h_in = h if h.requires_grad else h.requires_grad_(True)
            V = self.V_theta(xi_now, h_in).sum()
            grad_V, = torch.autograd.grad(
                V, h_in, create_graph=self.training, retain_graph=True)
            f = -grad_V
```

```

v = (v + dt * f / m_b) / (1.0 + dt * gamma)
h_new = h_in + dt * v
if cfg.ln_after_step:
    h_new = self._project(h_new)      # <-- the only new line
h = h_new
if return_trajectory:
    traj_h.append(h.detach().cpu())

return h, traj_h, traj_xi

```

Note that the projection is applied both at initialization — `h = self._project(emb)` before the loop — and after each step, so the trajectory lives on the shell from `h_0` onward. The `xi` fed to the force is still the causal cumulative mean of `h.detach()` (the leak-free SARF context), and the force is still the negative gradient of the same shared `V_theta`: LayerNorm changes where the state is allowed to live, not how the force is computed.

The rationale is geometric. A free scalar potential `V_theta` on all of $\mathbb{R}^d$ need not attain a minimum — its basins can run off to infinity, and the damped flow then has no well-defined attractor to settle into. Restricting `h` to the unit-LayerNorm shell makes the domain compact (it is S^{d-1} up to the mean-shift), and by the extreme-value theorem any continuous `V_theta` attains a finite minimum on it. LayerNorm-after-step is thus the cheapest way to guarantee a finite energy minimum — a genuine attractor basin — without altering the functional form of `V_theta` or adding any loss-side gauge term. It is also exactly the normalization every attention transformer already applies, which keeps the comparison to standard architectures clean.

This compactness carries a measurable cost on the oracle fit. Because `_project` is a non-affine map, the per-step displacement $\Delta h_1 = \text{project}(h_1 + dt \cdot v_{1+1}) - h_1$ is no longer an exact linear combination of `v_1` and `-grad_h V_theta`. The two-scalar oracle therefore cannot reach a perfect score on this variant:

`splm_oracle_fit.py` reports $R^2 \sim 0.931$ for the LN model versus $R^2 = 1.0000$ for the projection-free base model (see A Note on the oracle fit). The 0.931 is not a failure of conservativity — the force is still the exact negative gradient of a single shared potential — but the signature of the shell's curvature folded into each step.

Checkpoints written by the LN trainer tag themselves `variant="sarf_mass_ln"`, which is how `extract_oracle_tuples` and the trajectory extractors detect the projection (`has_ln = hasattr(model, '_project')`) and `cfg.ln_after_step`) and faithfully replay the projected dynamics during analysis.

Module `train_splm.py`

The four modules documented so far define the conservative architecture; the remaining modules in this appendix form the pipeline that trains it and then interrogates the geometry of the states it produces. The order below follows that pipeline: `train_splm.py` produces a checkpoint, `trajectory_extraction.py` turns the checkpoint into per-sentence hidden-state trajectories, and `shared_potential_fit.py`, `jacobian_symmetry.py` and `token_direction_fit.py` are the geometric diagnostics that consume those trajectories. `causal_probe.py` closes the loop with the regression test that guarantees the dynamics never leaks future tokens into the past.

`train_splm.py` is the reference training loop for the base `ScalarPotentialLM`. It is deliberately minimal — a single cross-entropy objective, AdamW, and a cosine schedule — because the architecture, not the optimiser, is the object of study. Three modes share one code path and differ only in model size and step budget: `smoke` (a 300-step pipeline check), `shakespeare` (the paper-grade Tiny-Shakespeare run), and `tinystories` (a larger corpus). Each run writes a checkpoint, a JSONL metric log, a loss curve and a Markdown summary under `results/`.

Per-mode configuration: `build_config`

`build_config` returns a `(model_cfg, train_cfg)` pair. The paper-grade `shakespeare` mode fixes the same eight-step, `d = 128` potential model used throughout the experiments:

```

elif mode == "shakespeare":
    model_cfg = SPLMConfig(
        vocab_size=50257, d=128, max_len=256,
        v_hidden=512, v_depth=3, L=8,
        init_m=1.0, init_gamma=1.0,
    )
    train_cfg = dict(
        batch_size=16, block_size=128,
        steps=4000, lr=5e-4, weight_decay=0.01,
        warmup_steps=200, grad_clip=1.0,
        eval_interval=200, eval_iters=40,
        log_interval=50,
    )

```

Note the small step budget (4000) and global gradient clip (`grad_clip = 1.0`). Both are conservative choices forced by the second-order nature of the model: every forward pass already builds a graph through `torch.autograd.grad(..., create_graph=True)` to compute the conservative force, so the backward pass differentiates through that inner gradient, and the loss landscape is correspondingly stiffer than for a plain decoder.

Learning-rate schedule: `lr_schedule`

A textbook linear warmup followed by cosine decay to zero — kept explicit rather than delegated to a scheduler object so the exact shape is auditable:

```

def lr_schedule(step: int, lr: float, warmup: int, total: int) -> float:
    if step < warmup:
        return lr * (step + 1) / warmup
    progress = (step - warmup) / max(total - warmup, 1)
    return lr * 0.5 * (1.0 + math.cos(math.pi * min(progress, 1.0)))

```

The training step

The inner loop is ordinary except for one consequence of the architecture: `model(x, y)` returns (`logits`, `loss`), and the forward pass that produced `loss` has already invoked `autograd` internally to integrate the dynamics. Calling `loss.backward()` therefore performs second-order backpropagation. Per-step gradients are clipped before the optimiser step, and the resulting `grad_norm`, together with the learned scalars `m` and `gamma`, is logged:

```

for step in range(train_cfg["steps"]):
    lr_now = lr_schedule(step, train_cfg["lr"],
                        train_cfg["warmup_steps"], train_cfg["steps"])
    for g in optim.param_groups:
        g["lr"] = lr_now

    xb, yb = get_batch(train_ids, train_cfg["batch_size"],
                        train_cfg["block_size"], rng)
    x = torch.from_numpy(xb).to(device)
    y = torch.from_numpy(yb).to(device)

    _, loss = model(x, y)
    optim.zero_grad(set_to_none=True)
    loss.backward()
    grad_norm = nn.utils.clip_grad_norm_(model.parameters(),
                                         train_cfg["grad_clip"])
    optim.step()

```

Two details are easy to miss but matter for reproducibility. First, evaluation cannot run under a blanket `torch.no_grad()`: the conservative force needs autograd even at inference, so `evaluate` is decorated `@torch.no_grad()` for memory but re-opens an inner with `torch.enable_grad()`: around the forward call:

```
@torch.no_grad()
def evaluate(model, ids, iters, batch_size, block_size, rng, device):
    model.eval()
    losses = []
    for _ in range(iters):
        xb, yb = get_batch(ids, batch_size, block_size, rng)
        x = torch.from_numpy(xb).to(device)
        y = torch.from_numpy(yb).to(device)
        with torch.enable_grad():          # the SPLM force needs autograd at eval too
            _, loss = model(x, y)
        losses.append(loss.item())
    model.train()
    return float(np.mean(losses))
```

Second, on CUDA the trainer disables TF32 by default. The second-order `create_graph=True` path is sensitive to TF32's 10-bit mantissa, so unless `--allow-tf32` is passed the matmuls are forced to true fp32:

```
torch.backends.cuda.matmul.allow_tf32 = False
torch.backends.cudnn.allow_tf32 = False
```

Module `trajectory_extraction.py`

`trajectory_extraction.py` is the bridge between a trained checkpoint and the geometric diagnostics. For each sentence in a small fixed corpus it re-runs the model with `return_trajectory=True`, capturing the full per-layer hidden-state path `h_0`, `h_1`, ..., `h_L`, the per-token next-token cross-entropy, and a flat per-token mass. The output is a list of `Trajectory` objects pickled to disk — deliberately the same dataclass the earlier E-init pipeline consumed, so every downstream fit reads one format.

Auto-detecting the checkpoint variant: `_build_model_from_checkpoint`

A single extractor must handle all three SPLM families (base, SARF-mass, and LayerNorm). It dispatches purely on the shape of the stored config dict — the presence of `ln_after_step` or `mass_mode` keys — then filters the dict down to the fields the chosen config dataclass actually accepts, so a checkpoint carrying extra bookkeeping keys still loads:

```
raw_cfg = dict(ck["model_cfg"])
is_ln = "ln_after_step" in raw_cfg
is_sarf = "mass_mode" in raw_cfg

if is_ln:
    from model_ln import SPLMSARFMassLNConfig, ScalarPotentialLMSARFMassLN
    CfgCls = SPLMSARFMassLNConfig
    ModelCls = ScalarPotentialLMSARFMassLN
elif is_sarf:
    from model_sarf_mass import SPLMSARFMassConfig, ScalarPotentialLMSARFMass
    CfgCls = SPLMSARFMassConfig
    ModelCls = ScalarPotentialLMSARFMass
else:
```

```
CfgCls = SPLMConfig
ModelCls = ScalarPotentialLM
```

```
accepted = {f.name for f in dataclasses.fields(CfgCls)}
filtered = {k: v for k, v in raw_cfg.items() if k in accepted}
cfg = CfgCls(**filtered)
model = ModelCls(cfg).to(device)
model.load_state_dict(ckpt["model_state_dict"])
model.eval()
```

A small robustness trick handles the logfreq mass mode: if the surprisal table referenced by the config is not on disk, the extractor writes a zero-filled dummy of the right length so the buffer shape matches, knowing that `load_state_dict` will immediately overwrite it with the trained values. Extraction never needs the original corpus file.

Extracting one trajectory: `extract_one`

The same `@torch.no_grad()` / inner `enable_grad()` pattern from the trainer reappears here, for the same reason. The trajectory tensors are stacked into a single $(L+1, T, d)$ array and the per-token loss is the shifted-target cross-entropy:

```
@torch.no_grad()
def extract_one(model, tokenizer, sentence, domain, split, device, max_len):
    ids = tokenizer.encode(sentence)[:max_len]
    x = torch.tensor(ids, dtype=torch.long, device=device).unsqueeze(0) # (1, T)

    with torch.enable_grad():
        logits, _, traj_list = model(x, return_trajectory=True)
        # traj_list: [h_0, h_1, ..., h_L], each (1, T, d)
        hs = torch.stack([t.squeeze(0) for t in traj_list], dim=0).numpy() # (L+1, T, d)

        if len(ids) >= 2:
            tgt = torch.tensor(ids[1:], device=device)
            ptl = torch.nn.functional.cross_entropy(
                logits[0, :-1, :], tgt, reduction="none"
            ).cpu().numpy()
        else:
            ptl = np.zeros((0,)), dtype=np.float32)
    ...
```

After extraction, every trajectory is centered per sentence. The centered state $x_{ps} = hs - \text{mean}_t(hs)$ is the quantity all three fits operate on; subtracting the per-sentence mean removes the trivial common-mode component and matches the convention of the original depth-direction pipeline:

```
for tr in trajs:
    tr.mu_ps = tr.hs.mean(axis=1, keepdims=True)
    tr.x_ps = tr.hs - tr.mu_ps
```

Module `shared_potential_fit.py`

This is the strict conservative-dynamics test. The Jacobian-symmetry diagnostic below asks only that each layer's local operator be symmetric; the shared-potential fit asks the much stronger question of whether a single smooth

scalar V_{psi} , shared across every layer, can explain all layers' forces at once. The ansatz, fit jointly over all layers, all tokens and all sentences, is

$$\Delta x_l \approx \alpha_l \cdot v_l - \beta_l \cdot \text{grad}_h V_{\text{psi}}(x_l)$$

with $v_l = x_l - x_{l-1}$ the velocity proxy, $\Delta x_l = x_{l+1} - x_l$ the displacement target, and α_l , β_l per-layer scalars absorbing scale. By construction an SPLM trajectory fits well — its true dynamics is the gradient of one scalar. The open question is whether GPT-2 trajectories admit any such shared description. The script reports three nested baselines: a static null ($\Delta x = 0$), velocity-only ($\Delta x = \alpha_l v_l$), and the full velocity-plus-shared- v fit.

The shared scalar network: `SharedPotential`

A small GELU MLP $\mathbb{R}^d \rightarrow \mathbb{R}$ whose gradient supplies the conservative force. `grad_V` returns dV/dh , building a differentiable graph only while training so the per-layer scalars can be co-optimised:

```
class SharedPotential(nn.Module):
    def __init__(self, d: int, hidden: int = 256, depth: int = 2):
        super().__init__()
        layers: List[nn.Module] = [nn.Linear(d, hidden), nn.GELU()]
        for _ in range(depth - 1):
            layers += [nn.Linear(hidden, hidden), nn.GELU()]
        layers += [nn.Linear(hidden, 1)]
        self.net = nn.Sequential(*layers)

    def forward(self, h: torch.Tensor) -> torch.Tensor:
        return self.net(h)

    def grad_V(self, h: torch.Tensor) -> torch.Tensor:
        """Return dV/dh of shape (N, d)."""
        h = h.requires_grad_(True) if not h.requires_grad else h
        V = self.forward(h).sum()
        g, = torch.autograd.grad(V, h, create_graph=self.training)
        return g
```

Pooling samples across layers: `build_samples`

Every interior layer of every trajectory contributes one (x, v, dx, layer) tuple per token. The loop runs l in $[1, L-1]$ because the velocity proxy needs $l-1$ and the displacement target needs $l+1$:

```
for tr in trajs:
    for ell in range(1, L):
        # need l-1 for v, l+1 for dx
        x_prev = tr.x_ps[ell - 1] # (T, d)
        x = tr.x_ps[ell]
        x_next = tr.x_ps[ell + 1]
        Xs.append(x)
        Vs.append(x - x_prev)
        Ys.append(x_next - x)
        Ls.append(np.full((x.shape[0],), ell, dtype=np.int64))
```

The joint fit: `fit_shared_V`

The per-layer scalars start at zero, so the model begins as the static null and must earn every increment of explained variance. Each step differentiates the shared potential at the batch positions, forms the prediction

$\alpha[l]*v - \beta[l]*\text{grad } v$, and minimises the MSE residual plus a mild penalty on the gradient magnitude to prevent blow-up:

```
alpha = nn.Parameter(torch.zeros(L, device=device))    # start at 0 -> velocity-only disabled
beta  = nn.Parameter(torch.zeros(L, device=device))
optim = torch.optim.AdamW(
    list(model.parameters()) + [alpha, beta],
    lr=lr, weight_decay=weight_decay, betas=(0.9, 0.95),
)

for step in range(steps):
    idx = rng.integers(0, N, size=batch_size)
    xb, vb, yb, lb = X_t[idx], V_t[idx], Y_t[idx], L_t[idx]

    xb_r = xb.clone().requires_grad_(True)
    Vout = model(xb_r).sum()
    gV, = torch.autograd.grad(Vout, xb_r, create_graph=True)

    y_pred = alpha[lb].unsqueeze(-1) * vb - beta[lb].unsqueeze(-1) * gV
    loss = F.mse_loss(y_pred, yb)
    reg = 1e-5 * torch.mean(gV ** 2)
    (loss + reg).backward()
    torch.nn.utils.clip_grad_norm_(list(model.parameters()) + [alpha, beta], 5.0)
    optim.step()
    optim.zero_grad(set_to_none=True)
```

All R^2 numbers are reported on held-out sentences, both per layer and pooled, and saved alongside a figure and a Markdown table. The headline quantity is the per-layer gain of the shared- v fit over velocity-only: a positive gain on test sentences is the signature of genuine shared conservative structure rather than mere momentum.

Module `jacobian_symmetry.py`

The Jacobian-symmetry test is the structural positive control. For a conservative flow $x_{l+1} - x_l \sim -dt * \text{grad}_h v(x_l) / m$, the local linearisation is the Hessian of a scalar and is therefore symmetric. So if we fit a per-step linear operator $y \sim M_l x_l + b_l$ on a PCA subspace, constraining $M_l = M_l^T$ should cost almost nothing when the dynamics is conservative. For the attention transformers in the companion failure study the unconstrained fit sits at R^2 in $[0.5, 0.8]$ while the symmetric fit is *negative* — a decisive failure of conservativity. This script runs the identical diagnostic on SPLM trajectories.

Unconstrained vs symmetric fit: `fit_linear_and_symmetric`

The unconstrained branch is a ridge-regularised least squares. The symmetric branch is the interesting one: it parameterises M by its upper-triangular free entries and builds a design matrix Φ so that solving the normal equations directly returns a symmetric operator. Each off-diagonal free entry (i, j) feeds *both* output column i (via predictor j) and column j (via predictor i), which is exactly the constraint $M[i, j] = M[j, i]$:

```
# ---- Unconstrained ridge fit.
G = Xc.T @ Xc          # (k, k)
reg = ridge * (np.trace(G) / max(N, 1)) * np.eye(k)
M_full = np.linalg.solve(G + reg, Xc.T @ Yc).T
r2_full = 1.0 - float(np.sum((Yc - Xc @ M_full.T) ** 2)) / (ss_tot + 1e-12)

# ---- Symmetric-constrained fit: parameterise M by its free upper triangle.
```

```

idx_ij = [(i, j) for i in range(k) for j in range(i, k)]
p      = len(idx_ij)
Phi = np.zeros((N * k, p))
for col, (i, j) in enumerate(idx_ij):
    if i == j:
        Phi[i * N:(i + 1) * N, col] = Xc[:, i]
    else:
        Phi[j * N:(j + 1) * N, col] = Xc[:, i] # column j gets predictor i
        Phi[i * N:(i + 1) * N, col] = Xc[:, j] # column i gets predictor j
vecY = np.concatenate([Yc[:, c] for c in range(k)], axis=0)
h_sym = np.linalg.solve(Phi.T @ Phi + reg2, Phi.T @ vecY)

```

Why velocity matters: `fit_second_order`

A position-only fit is confounded for damped second-order dynamics: the single-step transition mixes position x_1 with the hidden velocity $v_1 = x_1 - x_{1-1}$, and that mixing can manufacture apparent asymmetry even in a genuinely conservative flow. The velocity-aware fit removes the confound by regressing on the concatenated design $[X \mid V]$, letting an unconstrained A absorb the velocity term while M — now the clean per-step spring matrix — is the operator whose symmetry is tested:

```

# ---- Unconstrained (concatenated design [X | V] of shape (N, 2k)).
Phi = np.concatenate([Xc, Vc], axis=1) # (N, 2k)
G = Phi.T @ Phi
reg = ridge * (np.trace(G) / max(N, 1)) * np.eye(2 * k)
B = np.linalg.solve(G + reg, Phi.T @ Yc).T # (k, 2k)
M_full = B[:, :k].copy() # the spring matrix
A_full = B[:, k:].copy() # the velocity coupling

```

The symmetric variant reuses the upper-triangle construction for M while keeping A unconstrained. The reported verdict is the maximum test-set gap between the unconstrained and symmetric fits in the velocity-aware variant: a gap below 0.10 means the per-step spring matrix is consistent with a symmetric Hessian, i.e. the depth dynamics is conservative in h .

Module `token_direction_fit.py`

The depth-direction diagnostics trace a hidden state across layers at a fixed token. But autoregressive inference advances a second axis — token position — and that is the natural time axis of the Geodesic Hypothesis. `token_direction_fit.py` runs the very same two diagnostics with the token axis playing the role of time at a fixed layer. Its key design feature is reuse: it imports the scalar network and the fitting routines verbatim from the depth-direction modules, guaranteeing an apples-to-apples comparison:

```

from shared_potential_fit import SharedPotential
from jacobian_symmetry import (
    fit_linear_and_symmetric,
    fit_second_order,
    _pca_basis,
)

```

Building token triples: `build_token_triples`

At each layer e_{ll} (now including the embedding $e_{ll=0}$ and the final $e_{ll=L}$) and each interior token t , the script collects the triple $(h_t, v_t = h_t - h_{t-1}, dh_t = h_{t+1} - h_t)$. A `t_skip` margin drops the first and last few tokens of every sentence to avoid BOS/EOS initialisation transients:

```

for tr in trajs:
    T = tr.x_ps.shape[1]
    if T < 3 + 2 * t_skip:
        continue
    tlo = 1 + t_skip
    thi = T - 1 - t_skip
    for ell in range(L + 1):
        hs = tr.x_ps[ell] # (T, d)
        h_prev = hs[tlo - 1:thi - 1]
        h = hs[tlo:thi]
        h_next = hs[tlo + 1:thi + 1]
        Xs.append(h)
        Vs.append(h - h_prev)
        Ys.append(h_next - h)
        Ls.append(np.full((h.shape[0],), ell, dtype=np.int64))

```

The three-way reading is the payoff. Given the depth-direction result (SPLM uniform, matched-GPT decaying, pretrained GPT-2 collapsing in the middle layers), the token-direction test asks whether the same separator reproduces in sequence-time. If GPT-2 mid layers pass in token-time but fail in depth, the depth failure is a composition-of-operators artefact and the Geodesic Hypothesis survives in its natural coordinate; if they fail in both, no natural coordinate rescues the shared-scalar description for attention transformers. The module also emits a paper-headline JSON with full provenance for both sub-tests.

Module `causal_probe.py`

`causal_probe.py` is the regression test that protects the entire construction. A correct autoregressive model must satisfy a hard invariant: perturbing the token at position `t_pert` may change logits at positions `t >= t_pert`, but it must leave every logit at `t < t_pert` exactly unchanged. The probe perturbs one token, runs two forward passes, and measures the maximum logit change on the causal side.

This test exists because the SPLM `integrate()` method is vulnerable to a subtle leak. The conservative force is the gradient of a sum of potentials, $f(t) = -d(\sum_{t'} V[t']) / dh[t]$. When the context vector x_i is a causal running average that itself includes $h[t]$, differentiating the summed potential picks up off-diagonal terms $dV[t'] / dh[t]$ for $t' > t$, and a trained V_{θ} can route information backwards through that channel — inflating validation perplexity by orders of magnitude. The fix is one line: compute x_i from `h.detach()` so the autograd path from x_i back to h is severed. Every model config exposes a `causal_force: bool = True` field that controls it.

The probe: `causal_violation_probe`

Two inputs differ in a single token; the probe returns the maximum logit delta strictly before the perturbation (pre) and strictly after (post). For a causal model pre must be zero:

```

def causal_violation_probe(model, vocab_size, T=32, t_pert=20, seed=0):
    """Return (max delta on causal side, max delta on after-side, full delta vector)."""
    rng = np.random.default_rng(seed)
    xb = rng.integers(0, vocab_size, size=(1, T)).astype(np.int64)
    x_a = torch.from_numpy(xb)
    x_b = x_a.clone()
    orig = int(x_b[0, t_pert].item())
    x_b[0, t_pert] = (orig + 17) % vocab_size

    model.eval()
    out_a = model(x_a)

```

```

out_b = model(x_b)
logits_a = out_a[0].detach()
logits_b = out_b[0].detach()
diffs = (logits_a - logits_b).abs().max(dim=-1).values[0]
pre = float(diffs[:t_pert].max().item())
post = float(diffs[t_pert + 1:].max().item())
return pre, post, diffs

```

Regression discipline: `class_smoke`

With no checkpoint argument the script random-initialises every registered model class in both `causal_force=True` and `causal_force=False` modes and asserts that the fixed variant produces delta identically zero. Seeding before each construction makes the buggy/fixed pair share weights, so any difference is attributable solely to the xi detach:

```

for entry in _registry():
    torch.manual_seed(0)
    model_buggy = entry.cls_factory(entry.cfg_factory(False))
    torch.manual_seed(0)
    model_fixed = entry.cls_factory(entry.cfg_factory(True))
    pre_b, _, _ = causal_violation_probe(model_buggy, vocab_size=257)
    pre_f, _, _ = causal_violation_probe(model_fixed, vocab_size=257)
    ok_fixed = pre_f < 1e-6 # the fixed variant MUST give exactly 0
    verdict = "OK" if ok_fixed else "FAIL"

```

The registry itself is defensive. The core SPLM classes (`model_ln`, `model_sarf_mass`) are imported unconditionally, while every optional research variant is wrapped in try/except so the probe still runs against the shipped models even when an extension module is absent from this repository:

```

try:
    from model_multixi import ( # optional extension – not shipped here
        SPLMSARFmassLNMultiXiConfig, ScalarPotentialLMSARFmassLNMultiXi,
    )

    def _cfg_multixi(causal):
        return SPLMSARFmassLNMultiXiConfig(
            vocab_size=257, d=16, max_len=64, v_hidden=32, v_depth=2, L=4,
            mass_mode="global", ln_after_step=True,
            xi_channels=4, xi_alpha_inits=[0.0, 0.5, 0.9, 0.99],
            xi_learnable=True,
            causal_force=True if causal is None else causal,
        )
    out.append(ModelEntry(
        "multixi (K=4 EMAs)", _cfg_multixi, ScalarPotentialLMSARFmassLNMultiXi,
    ))
except Exception:
    pass

```

Probing natural transformers

The same probe applies to real transformers. A thin `_LogitsAdapter` wraps an arbitrary causal-LM — a local MatchedGPT checkpoint or a HuggingFace Hub model such as `gpt2` or `EleutherAI/pythia-160m` — so its forward returns the (logits, None) tuple the probe expects:

```

class _LogitsAdapter(torch.nn.Module):
    """Wrap an arbitrary causal-LM so forward returns (logits, None)."""
    def __init__(self, model, extract):
        super().__init__()
        self.model = model
        self._extract = extract

    def forward(self, x):
        return self._extract(self.model(x)), None

```

For these models the causal-side delta is zero by construction of the attention mask, so the natural-transformer probe is a sanity check on the harness rather than a test of the models — but running it through the same code path is what makes the SPLM result trustworthy: the conservative architecture is held to exactly the same causal standard as a standard decoder, and --strict turns any violation into a non-zero exit suitable for a CI gate.